SilkMind: Software and Symbolic AI, Learning Content V2.0

Team: SilkMind
Repository: https://github.com/skh0035/Mini-AI-repository
Rendered: 3 November 2026 from the `content/` folder (Markdown Content V2.0)
Contents: Module outline and development status · Topics 1–5 · Test bank · Glossary

Module Outline: Software & Symbolic AI

Team: SilkMind | **Version:** Markdown Content V2.0 | **Last updated:** 3 November 2026

This outline is the map of the whole module. It shows what learners will be able to do, the order in which topics are taught and why, what each topic contains, and how far each section has been developed. Terms are defined consistently in the glossary.

---

1. Who this module is for

Intended learners: first-year university students from any subject who have **no prior experience** of programming, computer science or artificial intelligence.

What we assume learners can already do (prerequisites):

Prerequisite	Why it is needed	Where it is covered if missing
Use a computer or phone: open apps, log in, browse the web	All examples are built on everyday apps (login screens, thermostats, spam filters, games)	Not covered; assumed
Read a simple table	Truth tables, trace tables and decision tables are used throughout	Topic 1 explains how to read a truth table
Basic comparisons with numbers (`<`, `>`, `>=`, `=`)	Rules such as `T < 18` and `links >= 4`	Topic 1, Sections 2.3 and 6 (Task B)
Everyday reasoning with "if … then …"	All rules are written in IF–THEN form	Built up in Topic 1

Not required: any programming language, maths beyond counting and comparing, or knowledge of how real AI models are trained. The two Python demos (`examples/`) are optional; every result they print is also shown in the text.

---

2. Intended learning outcomes

By the end of the module, learners should be able to:

#	Learning outcome	Taught in	Tested in (test bank)
LO1	Explain what software, an algorithm and a Boolean condition are, using an everyday example.	Topic 1	Q1–Q2
LO2	Write a simple decision as an IF–THEN rule and check it with a truth table.	Topic 1	Q3–Q5

#	Learning outcome	Taught in	Tested in (test bank)
LO3	Describe the parts of a rule-based (symbolic) AI system: facts, rules, knowledge base, working memory and inference engine.	Topic 2	Q6, Q10, Q11
LO4	Trace forward and backward chaining by hand to show how a rule-based system reaches and explains a conclusion.	Topic 2	Q7–Q9
LO5	Contrast symbolic AI with machine learning, trace a simple example of learning a rule from data, and choose a suitable approach for a problem.	Topic 3	Q12–Q16
LO6	Explain why symbolic AI rose, fell (the "AI winters") and changed, using key events as evidence.	Topic 4	Q17–Q20
LO7	Identify and trace rule-based reasoning in modern software, including systems that combine rules with machine learning.	Topic 5	Q21–Q25

---

3. Learning sequence and why it is in this order

```
Topic 1  Software & basic logic          "Computers decide with strict true/false
rules"
   │
   ▼
Topic 2  Rules & inference               "Symbolic AI = many such rules + an engine
that chains them"
   │
   ▼
Topic 3  Symbolic AI vs machine learning "Writing the rules yourself vs learning them
from data"
   │
   ▼
Topic 4  History: boom, AI winters, lessons  "Why the rules-only approach hit its
limits"
   │
   ▼
Topic 5  Symbolic AI in modern software  "Where rules are still used today, alone or
with ML"
   │
   ▼
Test bank  25 questions across LO1–LO7
```

Why this order:

- **Topic 1** gives the one idea everything else depends on: a computer decides by testing true/false conditions.
- **Topic 2** scales that idea up: a symbolic AI system is a large collection of IF–THEN rules plus a program that applies them automatically.
- **Topic 3** introduces machine learning only **after** learners understand rules. Its worked example shows that a learned model is *also* a rule; the difference is who writes it.

- **Topic 4** (history) comes after Topics 2–3 on purpose. The AI winters make sense once learners have seen, in Topic 2's activities, exactly how rule bases fail (gaps and conflicts). *(In V1.0, history came second; it was moved in V1.1.)*
- **Topic 5** closes the module by reusing every earlier idea (conditions, decision tables, forward chaining, learning + rules) on software learners use today.

How the topics connect (the "thread" activities):

Idea	Introduced	Reused in
Strict boundaries (`<` vs `<=`)	Topic 1, Task B (18 °C)	Topic 5, Activity B (5 km); Test bank Q4, Q23
AND vs OR mistakes	Topic 1, Task D	Test bank Q5
Gaps in a rule base	Topic 2, Part D	Topic 4 §4 (brittleness); Topic 5, Activity B
Rule conflicts	Topic 2, Part F	Topic 4 §4 (maintenance cost); Test bank Q19
Forward chaining	Topic 2 §4.3	Topic 5 §3 (knowledge graph); Topic 5 §4 (AlphaGeometry)
"A model is also a rule"	Topic 3 §3	Topic 3 §5; Topic 5 §4

---

4. Topic-by-topic plan and development status

Status labels used:

Label	Meaning
Outline only	Headings and plan, no learning text yet
Drafted	Explanations, worked example, activities and answers written; not yet reviewed by a second member
Checked	Reviewed by a member who did not write it; facts checked against sources; corrections made and recorded in `docs/review-log.md`
Needs revision	Reviewed; problems found that are not yet fixed

> **Status note (3 Nov 2026):** all core sections are now **Drafted** with full learning content. None is labelled **Checked** yet, because peer reviews and learner tests are scheduled for 4–6 November (see `docs/review-log.md`). The V1.1 and V2.0 drafts were prepared with AI assistance (see `docs/ai-use-log.md`), so peer review is required before any section is marked Checked.

Topic 1: Introduction to Software and Basic Logic

File: `01-software-and-logic.md` | **Outcomes:** LO1, LO2 | **Est. time:** 35 min

Subtopic	Example / activity	Status
2.1–2.2 Software, hardware, algorithms	Hardware/software table; recipe comparison	Drafted
2.3 Boolean conditions, AND/OR/NOT, truth tables	Operator table with everyday examples	Drafted
3 Worked example: login decision	Step-by-step trace, truth table, pseudocode	Drafted
4 Flowchart	Figure 1.1	Drafted
5–6 Activities A–D with answers	Thermostat algorithm, boundary truth table, AND stretch task, **find the error** (OR login rule)	Drafted
7 Common misunderstandings	4 misconceptions	Drafted
8 Check your understanding	4 questions with answers	Drafted

Topic 2: Rules and Inference: How a Symbolic AI Reasons ★ *core topic*

File: `02-rules-and-inference.md` | **Outcomes:** LO3, LO4 | **Est. time:** 55 min

Subtopic	Example / activity	Status
1–3 Symbols, facts, rules, knowledge base, working memory, inference engine	Detective comparison; Figure 2.1	Drafted
4.1–4.4 Forward chaining and explanations	Animal identifier; cycle-by-cycle trace table; Figure 2.2; "why?" chain	Drafted
4.5 Backward chaining	Same knowledge base, goal "penguin?"	Drafted
4.6 Code demo (optional)	`examples/topic02_mini_rule_engine.py`	Drafted; output checked
5–6 Activities A–F with answers	Trace, extend, backward chain, find the gap, **find the error in a trace**, **predict a rule conflict**	Drafted; answers checked with demo
7 Common misunderstandings	5 misconceptions	Drafted
8 Real expert systems and limits	DENDRAL, MYCIN, XCON; optional uncertainty box	Drafted; XCON re-sourced

Topic 3: Symbolic AI vs Machine Learning

File: `03-symbolic-ai-vs-machine-learning.md` | **Outcome:** LO5 | **Est. time:** 40 min

Subtopic	Example / activity	Status
1 Top-down vs bottom-up; ML terms	Term table; Figure 3.1	Drafted
2 Spam filter, two ways	Rule-based vs learned	Drafted
3 Worked example: learning a threshold rule from 10 labelled emails	Mistakes table for every k; predictions; `examples/topic03_learn_a_rule.py`	Drafted; checked with demo (**new in V2.0**)
4 Comparison table	6 criteria	Drafted
5 Common misunderstandings	4 misconceptions	Drafted
6–7 Activities A–D with answers	Retrain with new data, predict and explain, choose an approach, **find the error** in an accuracy claim	Drafted; Activity A checked with demo

Topic 4: History: The Rise, the AI Winters and the Lessons

File: `04-history-and-ai-winters.md` | **Outcome:** LO6 | **Est. time:** 35 min

Subtopic	Example / activity	Status
1–2 AI winter; timeline 1956–1993	Figure 4.1 timeline graphic (**new in V2.0**); event table	Drafted; dates re-checked
3 Worked example: explaining the first winter step by step	Expectations / technology / effect method; combinatorial explosion	Drafted
4 Second winter linked to Topic 2's limits	Table linking Parts D and F to industry problems	Drafted
5 Common misunderstandings	4 misconceptions	Drafted
6–7 Activities A–D with answers	Order events, explain, predict, **check a claim against sources**	Drafted

Topic 5: Symbolic AI in Modern Software

File: `05-symbolic-ai-in-modern-software.md` | **Outcome:** LO7 | **Est. time:** 40 min

Subtopic	Example / activity	Status
1 Forms, business rules, decision tables, game characters	Sourced examples (MDN, Salesforce, game-design lecture)	Drafted; sources added in V2.0
2 Worked example: tracing a game guard	Five-moment state trace	Drafted (**new in V2.0**)

Subtopic	Example / activity	Status
3 Knowledge graphs + worked chaining example	Registan graph; rule L1	Drafted
4 Neuro-symbolic AI	AlphaGeometry case study	Drafted
5 Common misunderstandings	4 misconceptions	Drafted
6–7 Activities A–D with answers	Trace the guard, **find the error in a decision table**, follow the graph, spot rules in an app	Drafted

Test bank and glossary

File	Content	Status
`06-test-bank.md`	25 questions mapped to LO1–LO7 (6 recall, 8 apply, 11 analyse), with answers and explanations	Drafted; tracing answers checked with demos
`glossary.md`	38 terms with the single definition used across all resources	Drafted

Optional extensions (clearly not core)

Extension	Where	Status
Uncertain rules (confidence values)	Topic 2, box in §8	Drafted (short box; not tested)
How real ML models are trained (beyond the threshold example)	Topic 3	Not planned for this module

---

5. Known gaps and how we will resolve them

#	Gap	Plan	Owner / date
1	No section is **Checked** yet	Peer review of every file by a non-author + one learner test per topic, recorded in `docs/review-log.md`	See review assignments; 4–6 Nov
2	Some Topic 4 facts (DARPA 1974, LISP machines 1987, Strategic Computing 1988) and XCON's rule count have only one source (Wikipedia)	Reviewer to find a second source (e.g. Russell & Norvig textbook history chapter in the library); otherwise soften wording to "about" / "by one account"	Topic 4 reviewer; 6 Nov
3	The learner tests may show explanations that are too fast for beginners	Revise the affected sections and record before/after in `review-log.md`	Topic authors; 6 Nov

#	Gap	Plan	Owner / date
4	Study Guide, slides and videos are being built in parallel	Any error found in production is fixed here first, then in every resource (see `docs/resource-mapping.md`)	All; ongoing

---

6. From Markdown to the three learning resources

The full section-by-section mapping is in `docs/resource-mapping.md`.

Topic 1: Introduction to Software and Basic Logic

Level: Beginner | **Estimated time:** 35 minutes | **Status:** Drafted V2.0 (awaiting peer review) **Comes before:** Topic 2: Rules and Inference

Learning objectives. After this topic you will be able to:

1. Explain what software is and how it differs from hardware.
2. Define an algorithm and a Boolean condition in your own words.
3. Trace a simple decision (a login check) step by step and check it with a truth table.
4. Write a simple decision as an IF–THEN rule.

---

1. Why start here?

The rest of this module is about **symbolic AI**: AI that works by following rules that people write down. To understand how such a system "thinks", you first need to understand how *any* program makes a decision. This topic gives you that foundation in about half an hour.

---

2. Concept explanation

2.1 Software and hardware

A computer is a machine. On its own, it does nothing useful. It needs instructions, and those instructions are called **software**.

> **Software:** the instructions (programs) that tell a computer what to do. The word was coined to distinguish these instructions from **hardware**, the physical parts of a computer (Britannica: Software).

You cannot touch software. You *can* touch hardware: the screen, the processor chip, the battery, the keyboard.

Hardware (physical)	Software (instructions)
Keyboard, screen, processor, memory chip	Operating system (e.g. Windows, Android)
Temperature sensor in a thermostat	The thermostat's control program
Phone camera	The camera app

Two common types of software are **system software**, which runs the computer itself (such as an operating system), and **application software**, which does a task for the user (such as a word processor or email app) (BBC Bitesize).

2.2 Computers follow instructions exactly

Computers cannot guess what you meant. They follow instructions exactly, one step at a time, in the order given. If an instruction is vague ("turn the heater on if it's a bit cold"), the computer has no way to ask what "a bit cold" means.

So programmers write instructions as precise, step-by-step procedures.

> **Algorithm:** a step-by-step procedure that solves a problem in a finite number of steps (Britannica: Algorithm). Each step must be clearly defined, and the procedure must eventually stop.

A recipe is a good everyday comparison: a list of clear steps, in order, with a finished dish at the end. A **program** is an algorithm written in a form a computer can run.

2.3 Decisions: true or false

Many steps in an algorithm are **decisions**, for example:

> IF the password is correct, THEN let the user in. OTHERWISE, refuse.

The question "is the password correct?" has only two possible answers: **true** or **false**. A question like this is called a **condition** (or **Boolean condition**).

> **Boolean logic:** a system of logic in which every statement is either true (1) or false (0), and statements are combined with the operators AND, OR and NOT. It is named after the English mathematician George Boole, who first set it out in 1847 (Britannica: Boolean algebra).

Operator	Meaning	Everyday example	True when…
AND	Both parts must be true	"Account exists" AND "password matches"	both are true
OR	At least one part must be true	"It is raining" OR "it is snowing"	either one (or both) is true
NOT	Flips the answer	NOT "account is locked"	the account is **not** locked

> **Truth table:** a table that lists every possible combination of true/false inputs and shows the result for each. It lets you check a rule completely, with no case forgotten (EBSCO Research Starters: Boolean algebra).

2.4 Why this matters for AI

The earliest approach to artificial intelligence, **symbolic AI**, is built from exactly these pieces. Experts write their knowledge down as many IF–THEN rules, and a program applies those rules automatically (Britannica: Expert systems). If you understand how one program makes one decision, you already understand the building block of a rule-based AI. Topic 2 shows how many rules are combined.

Key idea: Software = precise instructions. Many instructions are true/false decisions. Rule-based AI is built from the same kind of decisions.

---

3. Worked example: how does an email app decide whether you may log in?

Input (what the program receives): an email address and a password typed by the user.
Output (what the program produces): either "open the inbox" or an error message.

Step 1: Does an account exist for this email? The program looks up the email address in its list of accounts.

- Found → the condition is **true**. Continue to Step 2.
- Not found → the condition is **false**. Refuse access.

Step 2: Does the password match? The program compares the typed password with the one stored for that account.

- Match → **true**. Continue to Step 3.
- No match → **false**. Refuse access.

Note: in well-built systems, passwords are not stored as plain text. They are stored in a scrambled form ("salted and hashed"), and the program compares scrambled versions (NIST SP 800-63B-4). The true/false logic is the same.

Step 3: Decide and respond.

- If Step 1 AND Step 2 were both true → grant access and open the inbox.
- Otherwise → deny access.

The whole decision as one Boolean rule:

```
access_granted = (account_exists) AND (password_matches)
```

Checking the rule with a truth table:

Account exists?	**Password matches?**	**`account_exists AND password_matches`**	**Result**
True	True	True	**Access granted**

Account exists?	Password matches?	`account_exists AND password_matches`	Result
True	False	False	Access denied
False	True*	False	Access denied
False	False	False	Access denied

*In practice a password cannot match an account that does not exist. The row is included because a truth table always shows every combination. Only one row out of four leads to "granted", which is exactly how AND behaves.

The same logic written as pseudocode.

> **Pseudocode:** program-like steps written in plain language. It is not a real programming language; it is a way to write an algorithm clearly before (or instead of) coding it.

```
IF account_exists THEN
    IF password_matches THEN
        show inbox
    ELSE
        show "Invalid email or password"
ELSE
    show "Invalid email or password"
```

A design detail worth noticing. Both failure paths show the *same* message. This is deliberate. Security guidance recommends one generic error message whether the email or the password was wrong, so that attackers cannot use the login page to discover which email addresses have accounts (OWASP Authentication Cheat Sheet). Even this small program is a set of carefully chosen rules.

---

4. Visual: the login decision as a flowchart

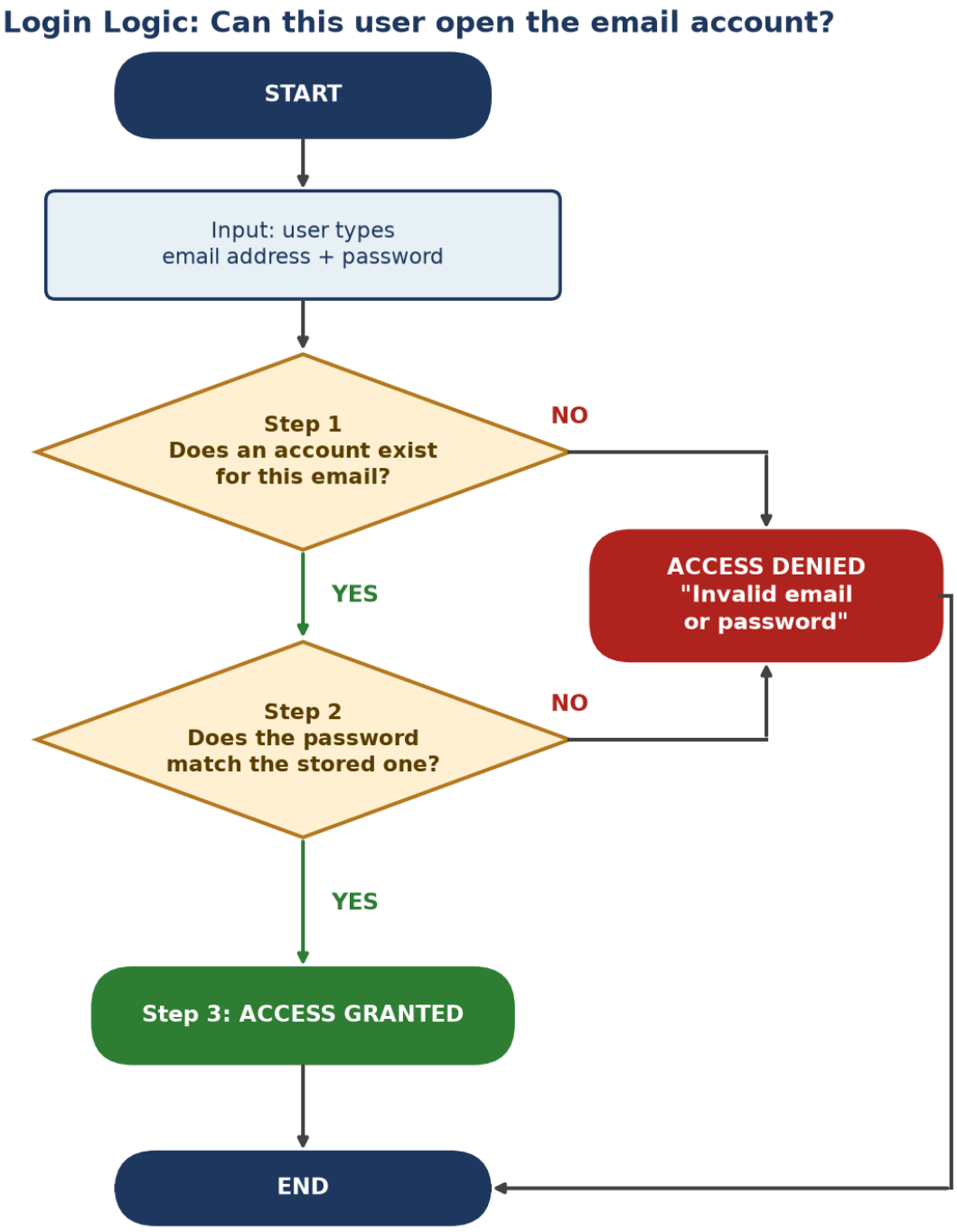


Figure 1.1. The login algorithm as a flowchart (added to the repository by @llex-27). Start at the top and follow the arrows. Rounded boxes are the start, end and outcomes; the rectangle is the input; each diamond is one Boolean test with exactly two exits (YES/NO). The only path to "Access granted" answers YES to both questions.

---

5. Learner activities: a thermostat rule and a faulty login rule

Scenario. You are writing the software for a smart thermostat. It has a temperature sensor and can switch a heater ON or OFF.

Task A (main task). Write a **3-step algorithm** that turns the heater on when the temperature is below 18 °C. Use the same pattern as the login example:

1. Get the input.
2. Test a true/false condition.
3. Act on the result, using **one IF–THEN–ELSE rule**.

Task B (truth table). Fill in the result column for three temperature readings: 15 °C, 18 °C and 21 °C.

Task C (stretch). Change your rule so the heater turns on only if the temperature is below 18 °C **and** someone is at home. Which Boolean operator did you use? Write the truth table for the new condition.

Task D (find the error). A student wrote this login rule:

```
access_granted = (account_exists) OR (password_matches)
```

Using a truth table, find one situation where this rule lets the wrong person in. Then correct the rule.

Hints: "below 18 °C" is a comparison, so it is either true or false for any reading. Decide what the heater should do when the condition is false. Write your answers before reading Section 6.

---

6. Expected outcome and answers

Task A: model answer

Step	What the software does	Matching step in the login example
1. Get the input	Read the current temperature from the sensor and call it `T`.	The user types an email and password.
2. Test the condition	Evaluate `T < 18`. The result is **true** or **false**.	"Does the account exist?" / "Does the password match?"
3. Act on the result	IF `T < 18` is true, THEN switch the heater **ON**; ELSE switch it **OFF**.	Grant or deny access.

```
T = read temperature sensor
IF T < 18 THEN
    heater = ON
ELSE
    heater = OFF
```

Why this is correct:

- It is an **algorithm**: a short, ordered list of clear steps that finishes with a result. A real thermostat would repeat it every few seconds.
- Step 2 is a **Boolean condition**: `T < 18` can only be true or false, so the program always knows what to do next.
- Step 3 is an **IF–THEN–ELSE rule**. It covers both outcomes, so the heater is never left in an unknown state.

Task B: truth table

Reading `T`	**`T < 18`**	**Heater**
15 °C	True	ON
18 °C	**False**	OFF
21 °C	False	OFF

The 18 °C row is the one most people get wrong. "Below 18" means *strictly less than* 18, so at exactly 18 °C the condition is false and the heater stays off. If you wrote `T <= 18`, you changed the rule. Computers need this kind of precision.

Task C: stretch answer

Operator: **AND**. Rule: `IF (T < 18) AND (someone_home) THEN heater = ON ELSE heater = OFF`.

`T < 18`	**`someone_home`**	**Heater**
True	True	**ON**
True	False	OFF
False	True	OFF
False	False	OFF

Task D: find the error

Account exists?	**Password matches?**	**`account_exists OR password_matches`**
True	True	True → granted ✔
True	**False**	**True → granted ✘**
False	True*	True → granted ✘
False	False	False → denied ✔

The second row is the dangerous one: **anyone who types a real user's email address gets in with any password**, because OR needs only one part to be true. The correct rule uses **AND** (Section 3): `access_granted = (account_exists) AND (password_matches)`.

Common mistakes to check for

1. **Skipping the input step.** The program must read the temperature before it can compare it.
2. **Writing a vague rule** such as "if it's cold, heat it". "Cold" is not a defined true/false condition.
3. **Forgetting the ELSE.** Without it, nothing says what happens when the condition is false.
4. **Mixing up AND and OR** (Task D). A quick truth table catches this every time.

---

7. Common misunderstandings

Misunderstanding	What is actually true
"OR means *one or the other, but not both*."	In Boolean logic, OR is true when **at least one** part is true, *including* when both are (truth table, Section 2.3). Everyday English sometimes means "not both", but computers do not.
"The computer understands what *password* means."	It only compares two pieces of stored data and gets true or false. All meaning comes from the people who wrote the rules. Topic 2 builds on this point.
"Pseudocode is a programming language."	It is plain-language steps for planning an algorithm; it cannot be run. Real programs must follow a language's exact rules.
"Software and the program are the same as the app icon on my screen."	The icon is just a shortcut. The software is the instructions behind it, stored in memory and run by the hardware.

---

8. Check your understanding

1. Is a phone's camera app hardware or software? What about the camera lens?
2. Why is "turn the heater on if it's chilly" *not* a good instruction for a computer?
3. `has_ticket = True`, `ticket_expired = True`. What is the result of `has_ticket AND NOT ticket_expired`?
4. A café gives a discount to customers who are students **or** over 65. Which operator is needed, and does a 70-year-old student get the discount?

Answers.

1. The app is software; the lens is hardware.
2. "Chilly" has no exact true/false meaning, so the computer cannot test it. A precise condition such as `T < 18` is needed.
3. `NOT ticket_expired` is False, so `True AND False` = **False**: the person cannot travel.
4. **OR**. Yes: OR is true when at least one part is true, and here both are.

---

9. Summary

- **Software** is the set of instructions; **hardware** is the physical machine.

- An **algorithm** is a finite list of clear steps; a program is an algorithm a computer can run.
- Decisions inside algorithms are **Boolean conditions** (true/false), combined with **AND**, **OR** and **NOT**, and checked with **truth tables**.
- An **IF–THEN rule** is the building block of rule-based (Symbolic) AI, which Topic 2 explains.

---

10. References

All links accessed on 21 October 2026 and re-checked on 31 October 2026.

1. Encyclopaedia Britannica. *Software.* https://www.britannica.com/technology/software
2. BBC Bitesize. *What is software?* (Computational constructs, National 4). https://www.bbc.co.uk/bitesize/guides/zcg9kqt/revision/1
3. Encyclopaedia Britannica. *Algorithm.* https://www.britannica.com/science/algorithm
4. Encyclopaedia Britannica. *Boolean algebra.* https://www.britannica.com/topic/Boolean-algebra
5. EBSCO Research Starters. *Boolean algebra.* https://www.ebsco.com/research-starters/science/boolean-algebra
6. Encyclopaedia Britannica. *Artificial intelligence: Expert systems.* https://www.britannica.com/technology/artificial-intelligence/Expert-systems
7. Temoshok, D., et al. (2025). *NIST Special Publication 800-63B-4, Digital Identity Guidelines: Authentication and Authenticator Management.* National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63B-4.pdf
8. OWASP Foundation. *Authentication Cheat Sheet.* https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Changes in V2.0: added Task D (find the error in an OR login rule) with answer, and Section 7 (common misunderstandings).

Changes from V1.0 to V1.1: activity wording now matches the model answer (V1.0 asked for "3 rules" but answered with 3 steps); terms *condition*, *truth table* and *pseudocode* are now defined where they first appear; added a hardware/software table, a truth-table task and a self-check section; removed references that were listed but not used (Britannica Kids, Plus Maths, Stanford Python notes, Crash Course) and replaced a lecture-notes PDF and Wikipedia with Britannica's expert-systems article.

Topic 2: Rules and Inference: How a symbolic AI Reasons

Level: Beginner | **Estimated time:** 55 minutes | **Status:** Drafted V2.0 (fully developed topic; awaiting peer review) **Builds on:** Topic 1: Software and Basic Logic | **Comes before:** Topic 3: Symbolic AI vs Machine Learning

Learning objectives. After this topic you will be able to:

1. Explain what "symbolic" means in symbolic AI.
2. Name and describe the parts of a rule-based AI system: facts, rules, knowledge base, working memory and inference engine.
3. Trace **forward chaining** by hand, cycle by cycle, and say when it stops.
4. Explain how the system can answer "why?" by showing the rules it used.
5. Give one real example of an expert system and one important limitation.

---

1. From one rule to many

In Topic 1 a program made **one** decision with **one** rule:

```
IF T < 18 THEN heater = ON ELSE heater = OFF
```

Now imagine a doctor's knowledge about blood infections, written down as **hundreds** of rules like this. A program that stores those rules and applies them automatically to a new patient can give advice that sounds like an expert's. That is the idea behind **symbolic AI** and its best-known product, the **expert system**.

> **Expert system:** a computer program that uses AI methods to solve problems in one specialised area, the kind of problem that normally needs a human expert (Britannica: Expert system).

---

2. What does "symbolic" mean?

> **Symbol:** a name or label that stands for something, for example `is a bird`, `has feathers`, `temperature_high`.

Symbolic AI represents knowledge as symbols that humans can read, and reasons by combining those symbols with rules.

One important point: **the computer does not know what the symbols mean.** To the program, `has feathers` is just a string of characters. It does not know that feathers are soft or

that birds have them. All the "meaning" comes from the rules people write. For example, the program does not know that `cannot fly` is the opposite of `can fly` unless a rule says so. Keep this in mind; it explains many of the limits in Section 8.

---

3. The parts of a rule-based system

Part	What it is	Everyday comparison
Fact	A statement taken as true for the current case, e.g. `has feathers`.	A clue in a detective story
Rule	An IF–THEN statement. The **IF part** (also called the *condition* or *antecedent*) is one or more facts joined by AND. The **THEN part** (*conclusion* or *consequent*) is a new fact.	"If the footprints are wet, the suspect came in from outside"
Knowledge base	The stored collection of rules for one subject area. It is written once, usually with the help of human experts.	The detective's experience
Working memory	The facts about *this particular case*: the starting facts plus any new facts the system works out.	The detective's notebook
Inference engine	The program that repeatedly finds rules whose IF part is true and adds their THEN part to working memory.	The detective's reasoning process

> **Inference:** working out a new fact from facts you already have. Britannica describes an expert system's two basic parts as a knowledge base and an **inference engine** that draws conclusions from it (Britannica: Expert systems).

> **A rule "fires"** when the inference engine finds that its IF part is true and adds its THEN part to working memory.

Figure 2.1 Parts of a rule-based (symbolic) AI system

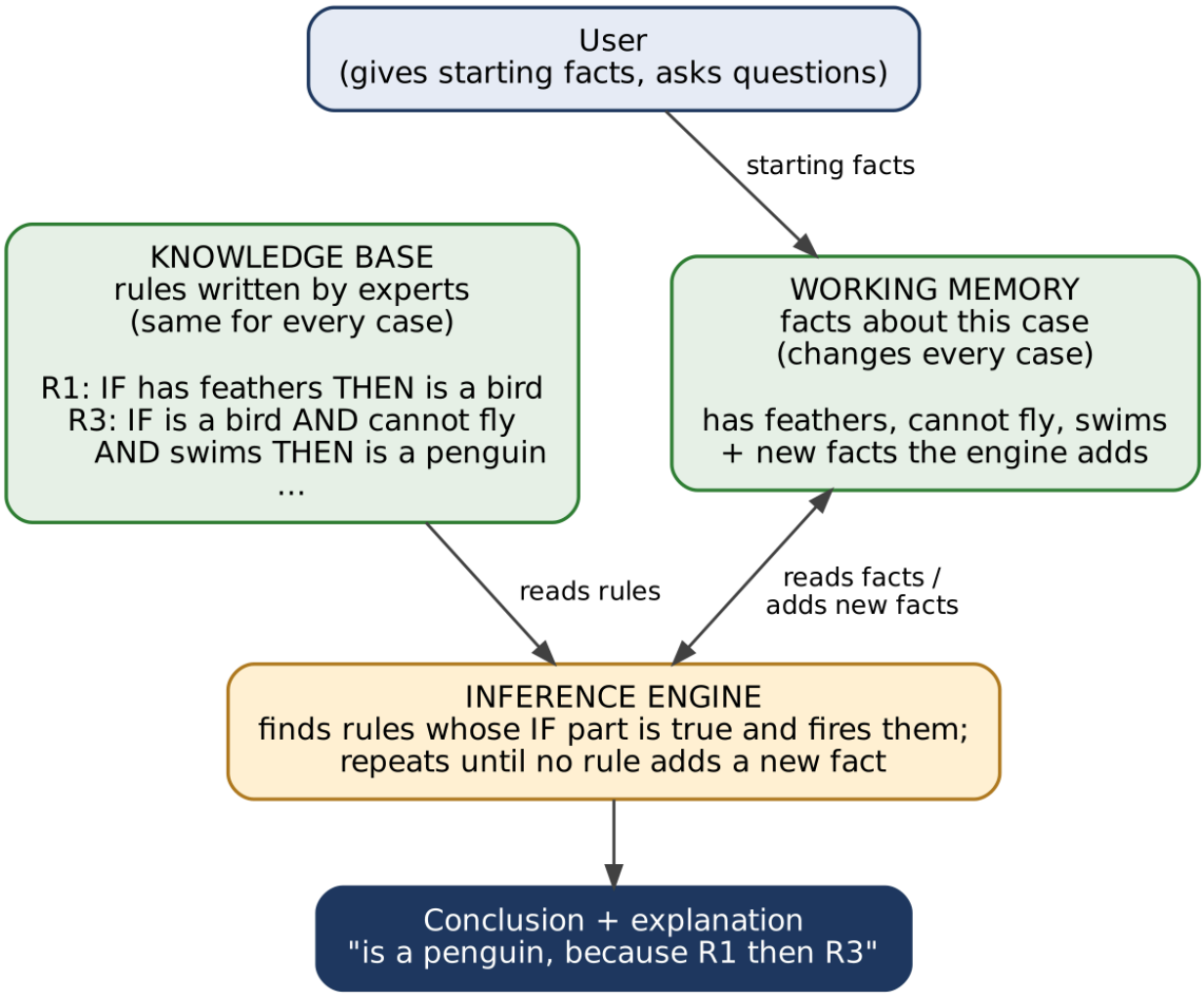


Figure 2.1. The parts of a rule-based system. The knowledge base stays the same for every case; working memory changes for each case. The inference engine is the only part that "does" anything. (Diagram source: `assets/diagram-sources/topic02-expert-system-architecture.dot`.)

---

4. Worked example: an animal identifier

We will build a tiny expert system that identifies an animal from a description. (Small animal-classification systems like this are a standard teaching example; see the frog example in Wikipedia: Forward chaining.)

4.1 The knowledge base (6 rules)

Rule	IF …	THEN …
R1	has feathers	is a bird
R2	gives milk	is a mammal

Rule	IF …	THEN …
R3	is a bird AND cannot fly AND swims	is a penguin
R4	is a bird AND can fly AND hunts at night	is an owl
R5	is a mammal AND eats meat AND has black stripes	is a tiger
R6	is a mammal AND has hooves AND has black stripes	is a zebra

Notice that R3–R6 use facts that are produced by R1 and R2. Rules can build on each other. This is called **chaining**.

4.2 The case

A user describes an animal. The starting facts in working memory are:

```
has feathers,  cannot fly,  swims
```

4.3 Forward chaining, step by step

> **Forward chaining:** the inference engine starts from the known facts and keeps firing rules to add new facts, until no rule can add anything new. It is called **data-driven** because the facts decide which rules fire (Wikipedia: Forward chaining).

The engine works in **cycles**. In each cycle it checks the rules in order (R1, R2, …), fires the **first** rule whose IF part is completely true **and** whose THEN part is not already known, then starts the next cycle.

Cycle 1.

- R1: IF *has feathers*? Yes, it is in working memory → **R1 fires**. Add `is a bird`.

Cycle 2. Working memory now: `has feathers, cannot fly, swims, is a bird`.

- R1: IF part true, but `is a bird` is already known → skip (it would add nothing new).
- R2: *gives milk*? Not known → false.
- R3: *is a bird* ✔ AND *cannot fly* ✔ AND *swims* ✔ → **R3 fires**. Add `is a penguin`.

Cycle 3.

- R1, R3: true but add nothing new. R2, R4, R5, R6: at least one condition is not known → false.
- No rule can add a new fact → **STOP**.

Trace table (the standard way to record forward chaining):

Cycle	Working memory at start of cycle	Rules with a true IF part that would add something new	Rule fired	New fact
1	has feathers, cannot fly, swims	R1	**R1**	is a bird
2	… + is a bird	R3	**R3**	is a penguin
3	… + is a penguin	none	none	**stop**

Conclusion: the animal is a penguin.

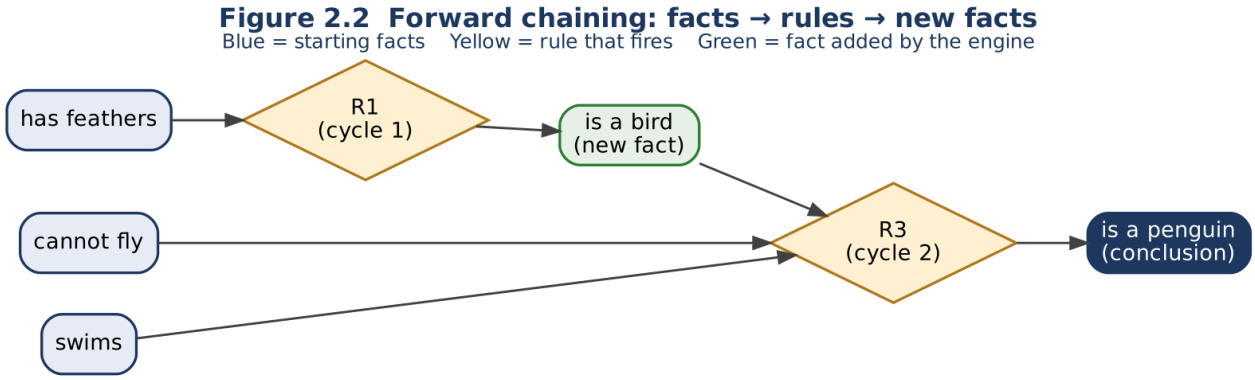


Figure 2.2. The same trace as a picture. Each rule takes facts in and produces one new fact; the new fact from cycle 1 becomes an input in cycle 2.

4.4 Asking "why?": explanations

Because every new fact was added by a named rule, the system can explain its answer by walking back through the chain:

```
Why "is a penguin"?   Because R3: is a bird AND cannot fly AND swims.
  Why "is a bird"?    Because R1: has feathers.
  "has feathers", "cannot fly", "swims" were given by the user.
```

This is one of the biggest strengths of symbolic AI. Britannica notes that expert systems can show the sequence of rules behind a conclusion, which helps users judge whether to trust it (Britannica: Expert system). The MYCIN medical system, for example, could explain its reasoning on request (Britannica: Expert systems).

4.5 The same idea the other way round: backward chaining

> **Backward chaining:** the engine starts from a **goal** ("Is it a penguin?") and works backwards, checking which facts would be needed to prove it. It is called **goal-driven** (Wikipedia: Forward chaining).

Goal: *is a penguin?*

1. Which rule concludes `is a penguin`? **R3**. So we need: `is a bird`, `cannot fly`, `swims`.
2. `is a bird`: not given, but **R1** concludes it if `has feathers`. Is `has feathers` known? Yes ✔.
3. `cannot fly`: known ✔. `swims`: known ✔.
4. All conditions are proved → goal is **true**.

If a needed fact is not known, a backward-chaining system can **ask the user** ("Does the animal swim?"). This is how many diagnosis systems hold a conversation: they ask only the questions that matter for the goal they are testing.

	Forward chaining	**Backward chaining**
Starts from	Facts	A goal (a hypothesis)
Question it answers	"What can I conclude from these facts?"	"Is this particular conclusion true?"
Good for	Monitoring, configuration, "what follows?"	Diagnosis, "is it X?", asking targeted questions

4.6 Optional: watch a computer do it

The file `examples/topic02_mini_rule_engine.py` contains the same six rules and a 20-line inference engine. Running it prints exactly the trace above:

```
Starting facts: has feathers, cannot fly, swims

Cycle 1: R1 fires because has feathers -> add 'is a bird'
Cycle 2: R3 fires because is a bird AND cannot fly AND swims -> add 'is a penguin'
Cycle 3: no rule adds a new fact -> STOP

Why is it a penguin?
- 'is a penguin' because of R3 (IF is a bird AND cannot fly AND swims)
    - 'is a bird' because of R1 (IF has feathers)
        - 'has feathers' was given as a starting fact
    - 'cannot fly' was given as a starting fact
    - 'swims' was given as a starting fact
```

You do not need to read Python for this module. The point is that the computer's "reasoning" is nothing more than the loop you just did by hand.

---

5. Learner activity

Use the knowledge base in Section 4.1 (R1–R6) and the same strategy: in each cycle, fire the first rule (in order R1→R6) whose IF part is true and whose THEN part is new.

Part A: trace it. The starting facts are: `gives milk`, `has hooves`, `has black stripes`. Complete a trace table (cycle, working memory, rule fired, new fact). What does the system conclude, and in which cycle does it stop?

Part B: extend the knowledge base. Write one new rule, R7, that identifies a **giraffe**, using `is a mammal` plus two other facts of your choice. Then write the starting facts a user would need to give for R7 to fire.

Part C: backward chaining. The goal is "*is a tiger?*". Which facts would the system need to check, and in what order? Which question would it ask the user if it knew only `gives milk`?

Part D: find the gap. A user enters `has feathers` and `can fly`. What does the system conclude? Is that a mistake by the computer, or a limit of the knowledge base? Explain in one or two sentences.

Part E: find the error in a trace. A classmate traced the starting facts `gives milk`, `has black stripes`, `eats meat` like this:

Cycle	Rule fired	New fact
1	R5	is a tiger
2	none	stop

The final answer (tiger) happens to be right, but the trace is wrong. Explain the mistake and write the correct trace.

Part F: predict the outcome. Before tracing, predict what the system will conclude from `gives milk`, `eats meat`, `has hooves`, `has black stripes`. Then trace it. What does the result reveal about the knowledge base, and how could you fix it?

---

6. Expected outcomes and answers

Part A

Cycle	Working memory at start	Rule fired	New fact
1	gives milk, has hooves, has black stripes	**R2** (gives milk)	is a mammal
2	… + is a mammal	**R6** (is a mammal AND has hooves AND has black stripes)	is a zebra
3	… + is a zebra	none	**stop**

Conclusion: zebra, stopping in cycle 3. R5 (tiger) does **not** fire, because `eats meat` is not in working memory, even though two of its three conditions are true. AND needs *every* condition to be true (Topic 1). *You can check this answer by changing the starting facts in the example program.*

Part B (one possible answer)

```
R7: IF is a mammal AND has a very long neck AND has hooves THEN is a giraffe
Starting facts needed: gives milk, has a very long neck, has hooves
```

Any rule is acceptable if (1) it includes `is a mammal`, (2) the conditions are clear true/false facts, and (3) the starting facts include `gives milk`. Without it, R2 cannot produce `is a mammal`, and R7 can never fire. **Watch out:** if your giraffe rule used `has black stripes`, the same facts might also fire R6 and the system would conclude "zebra" too. Rules can conflict, which is why rule writers must check new rules against old ones.

Part C

1. The goal `is a tiger` is concluded only by **R5**, so the system needs `is a mammal`, `eats meat` and `has black stripes`.
2. `is a mammal` comes from **R2**, which needs `gives milk`. It is known ✔.
3. `eats meat` is not known, so the system would **ask: "Does the animal eat meat?"** (then "Does it have black stripes?").

The system asks only about facts that matter for the goal. It would never ask about feathers, because no step towards "tiger" needs them.

Part D

Cycle 1: R1 fires → `is a bird`. Cycle 2: no rule adds anything new → stop. The system concludes only **"is a bird"**, not which bird. R4 (owl) needs `hunts at night` as well, and no rule covers other flying birds.

This is **not** a computer error. The engine did exactly what the rules say. It is a **gap in the knowledge base**: the system knows only what its rules cover. This is the main weakness of rule-based systems, explored in Section 8 and in Topic 4.

Part E

R5 cannot fire in cycle 1, because one of its conditions, `is a mammal`, is **not yet in working memory**. It is only added when R2 fires. The correct trace is:

Cycle	Rule fired	New fact
1	**R2** (gives milk)	is a mammal
2	**R5** (is a mammal AND eats meat AND has black stripes)	is a tiger
3	none	stop

A correct-looking final answer does not prove the reasoning was correct. The order of firing matters because new facts only become available in the *next* cycle.

Part F

Cycle	Rule fired	New fact
1	R2	is a mammal
2	R5	is a tiger

Cycle	Rule fired	New fact
3	R6	is a zebra
4	none	stop

The system concludes that the animal is **both a tiger and a zebra**. The engine did nothing wrong; the **knowledge base** allows two contradictory conclusions because R5 and R6 overlap (both accept a striped mammal). This is called a **rule conflict**. Possible fixes: add a condition to R5 such as `does not have hooves`, or add a rule that rejects the case as impossible ("carnivores with hooves are not in this knowledge base"). Either fix must then be re-tested against Parts A and E.

Both answers were checked by running the example program with these starting facts.

Common mistakes

1. Firing a rule when only *some* of its AND conditions are true (e.g. firing R5 in Part A).
2. Firing the same rule again and again. A rule that adds nothing new does not count.
3. Forgetting that new facts (like `is a mammal`) can make other rules true in the *next* cycle (Part E).

---

7. Common misunderstandings

Misunderstanding	What is actually true
"The expert system understands animals."	It matches symbols. It would draw the same conclusions if every word were replaced by a code like `X17` (Section 2).
"If the answer is right, the reasoning is right."	Part E reached the right answer with an impossible trace. Explanations (Section 4.4) let us check the reasoning, not just the answer.
"If the system gives a wrong or strange answer, the inference engine has a bug."	Usually the engine is fine and the **knowledge base** is incomplete (Part D) or conflicting (Part F).
"Forward and backward chaining give different answers."	With the same rules and facts they reach the same conclusions; they differ in **direction** and in **which questions they ask** (Section 4.5).
"More rules always make the system better."	Each new rule can conflict with existing ones (Part B warning, Part F), so every addition must be re-tested.

---

8. Real expert systems, and their limits

System	When / where	What it did	Source
DENDRAL	From 1965, Stanford University (Feigenbaum and Lederberg)	Suggested the molecular structure of chemical compounds from spectrograph data; often called the first expert system	Britannica: Expert systems
MYCIN	From 1972, Stanford University	Diagnosed blood infections and recommended treatment using about 500 rules; could explain its reasoning; performed about as well as specialists and better than general practitioners	Britannica: Expert systems
XCON (R1)	Developed by John McDermott (Carnegie Mellon University) for Digital Equipment Corporation (DEC); used in DEC's manufacturing from January 1980	Checked customers' computer orders and chose the parts needed to make each system complete; by 1986 it was estimated to save DEC about $25 million a year. It grew to thousands of rules (about 2,500 by one account)	McDermott (1981, 1982); Middlesex University notes; Wikipedia: XCON

Strengths

- **Explainable:** every conclusion can be traced to named rules (Section 4.4).
- **Uses expert knowledge directly:** no large dataset is needed, only experts willing to state their rules.
- **Predictable:** the same facts always give the same answer.

Limits

- **No common sense, no awareness of its own limits.** Britannica gives a striking example: told that a patient was bleeding heavily from a gunshot wound, MYCIN would still try to find a bacterial cause, because bacteria were all its rules knew about (Britannica: Expert systems).
- **Knows only what its rules cover**, as Part D showed. Anything unexpected is either missed or handled wrongly. Systems that fail like this outside their narrow area are often called **brittle**.
- **Hard to maintain.** Every new situation needs new rules, and new rules can conflict with old ones (Part B's warning). XCON needed thousands of rules.
- **Uncertainty is hard.** Real expert knowledge is rarely 100% certain, so real systems attached confidence or probability values to rules (Britannica: Expert system). Our simple true/false version leaves this out.

These limits are a large part of why interest in rule-only AI collapsed in the late 1980s (Topic 4), and why most modern AI learns from data instead (Topic 3).

> **Optional extension: uncertain rules** *(not required for the activities or the test bank)* Real expert knowledge is rarely 100% certain. A doctor's rule is more like "IF these symptoms THEN *probably* this infection". Real systems therefore attached a confidence

> or probability value to rules (Britannica: Expert system), and the engine combined these values as it chained. This module keeps every fact strictly true or false, as in Topic 1, so that the reasoning can be traced by hand.

---

9. Check your understanding

1. Which part of an expert system changes for every new case: the knowledge base or working memory?
2. In forward chaining, when does the inference engine stop?
3. A rule is `IF A AND B THEN C`. Working memory contains `A` only. Does the rule fire?
4. A doctor wants to test one suspected illness by asking the patient only relevant questions. Forward or backward chaining?
5. Give one reason why an expert system's answer may be easier to trust than an answer with no explanation.

Answers.

1. **Working memory.** The knowledge base (rules) stays the same.
2. When a full cycle passes in which **no rule can add a new fact**.
3. **No.** AND requires both A and B to be true.
4. **Backward chaining.** It starts from a goal and asks only for the facts needed to prove it.
5. It can show the exact rules it used, so a human expert can check each step (Section 4.4).

---

10. Summary

- **Symbolic AI** stores knowledge as human-readable **symbols** and **IF–THEN rules**. The computer does not understand the symbols; all meaning comes from the rules.
- A rule-based system has a **knowledge base** (rules), **working memory** (facts for this case) and an **inference engine** (the loop that fires rules).
- **Forward chaining** goes from facts to conclusions until nothing new can be added; **backward chaining** starts from a goal and works back to the facts.
- Rule-based systems can **explain** every conclusion, but they are **brittle**: they know only what their rules cover.

---

11. References

Links accessed 31 October 2026; XCON sources added 3 November 2026.

1. Encyclopaedia Britannica. *Expert system.* https://www.britannica.com/technology/expert-system
2. Encyclopaedia Britannica. *Artificial intelligence: Expert systems* (sections on knowledge base and inference engine, DENDRAL, MYCIN, and limitations).

https://www.britannica.com/technology/artificial-intelligence/Expert-systems
3. Wikipedia. *Forward chaining* (definition; data-driven vs goal-driven; frog example). https://en.wikipedia.org/wiki/Forward_chaining
4. McDermott, J. (1981). R1: The formative years. *AI Magazine*, 2(2). (Confirms use at DEC since January 1980.) https://aitopics.org/doc/journals:E2B8C719
5. McDermott, J. (1982). R1: A rule-based configurer of computer systems. *Artificial Intelligence*, 19(1), 39–88. https://doi.org/10.1016/0004-3702(82)90021-2
6. Huyck, C. *Configuration with R1/XCon.* BIS2040 course notes, Middlesex University. (Reports the estimated $25M/year saving by 1986.) https://cwa.mdx.ac.uk/bis2040/lect6ES2/xCon.html
7. Wikipedia. *Xcon* (rule count). https://en.wikipedia.org/wiki/Xcon

Further reading (not used for specific claims):

- Microsoft. *AI for Beginners, Lesson 2: Knowledge Representation and Expert Systems.* https://github.com/microsoft/AI-For-Beginners/blob/main/lessons/2-Symbolic/README.md
- Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.), Pearson. Chapters on logical agents and inference. *(Standard university textbook; team to confirm chapter numbers from the library copy.)*

Note on sources: Wikipedia is used only for the general definition of forward/backward chaining and for XCON's rule count. In V2.0 the XCON deployment date and savings were confirmed in a second source (refs 4 and 6); the figure of about 2,500 rules is still supported only by Wikipedia, so the text says "by one account". The animal-identifier rules and all activity data are the team's own teaching material.

Topic 3: Symbolic AI vs Machine Learning

Level: Beginner | **Estimated time:** 40 minutes | **Status:** Drafted V2.0 (awaiting peer review) **Builds on:** Topic 1 (conditions, truth tables) and Topic 2 (rules, knowledge base) | **Comes before:** Topic 4: History

Learning objectives. After this topic you will be able to:

1. Describe the difference between **writing rules** (symbolic AI) and **learning from examples** (machine learning).
2. Trace, by hand, how a very simple machine-learning program "learns" a rule from labelled data.
3. Compare the two approaches on explainability, data needed, handling of unusual cases, and maintenance.
4. Choose a suitable approach for a problem and justify the choice.

---

1. Two ways to give a computer knowledge

In Topic 2, **people** wrote every rule (R1–R6). The computer only applied them. This is called a **top-down** approach: knowledge flows from human experts down into the program.

There is a second way. Instead of writing the rules, you give the computer **many examples together with the correct answers**, and a program finds the pattern itself. This is a **bottom-up** approach: the knowledge is built up from data.

> **Machine learning (ML):** a branch of AI in which software learns patterns from data, instead of following rules that a programmer wrote in advance (Britannica: Machine learning; IBM Developer: What is machine learning?).

Term	Meaning	In the spam example below
Labelled data	Examples that each come with the correct answer (the **label**)	10 emails, each marked "spam" or "not spam"
Feature	A measurable property of an example that the program is allowed to look at	The number of links in the email
Training	The process in which an ML program searches for the pattern that best fits the labelled data	Trying different rules and counting mistakes
Model	The result of training; the learned "rule", used to answer new cases	`IF links >= 4 THEN spam`
Prediction	The model's answer for a new, unlabelled example	Email K (7 links) → spam

Figure 3.1 Two ways to give a computer knowledge

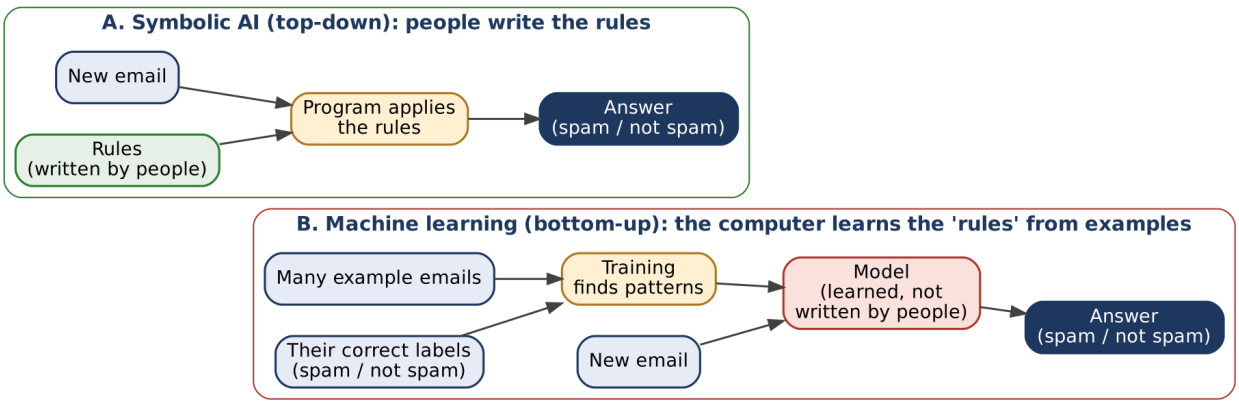


Figure 3.1. In symbolic AI, people supply the rules. In machine learning, people supply examples and answers, and training produces the model. Both then answer new questions.

---

2. Same problem, two approaches: a spam filter

Symbolic version. A programmer writes the rules:

```
R1: IF subject contains "FREE MONEY" THEN spam
R2: IF sender is in the user's contacts THEN not spam
R3: IF email contains more than 5 links AND sender is unknown THEN spam
```

- ✔ Easy to explain: "This was marked spam because of R1."
- ✘ Spammers quickly write "FR3E M0NEY", and R1 no longer matches. Someone must keep writing new rules.

Machine-learning version. The team collects emails that users have already marked as spam or not spam, and lets a program find the pattern. Section 3 shows, in a tiny version you can do by hand, what "finding the pattern" means.

(Both filters here are simplified teaching examples, not descriptions of a specific product. Real email services combine many techniques.)

---

3. Worked example: watching a computer "learn" a rule

Real ML models look at thousands of features at once and are trained with mathematics beyond this module. But the **core idea**, *try possible rules and keep the one that makes the fewest mistakes on the labelled examples*, can be shown with one feature and ten emails.

3.1 The labelled training data

Email	A	B	C	D	E	F	G	H	I	J
Number of links	0	1	1	2	3	2	4	5	6	8

Email	A	B	C	D	E	F	G	H	I	J
Label (correct answer)	not	not	not	not	not	**spam**	**spam**	**spam**	**spam**	**spam**

Notice that the data is **not perfect**: email F is spam with only 2 links, while email E is genuine with 3 links. Real data is always a bit messy like this.

3.2 The kind of rule the program is allowed to learn

We let the program learn **one** rule of this shape, where `k` is a number it must choose:

```
IF number_of_links >= k THEN spam ELSE not spam
```

Choosing `k` is the whole "training". (This shape is a **threshold rule**: everything at or above the threshold is classed as spam.)

3.3 Training: try each k and count mistakes

Step 1: try k = 2. The rule says "2 or more links → spam".

Email	Links	Rule predicts	Correct label	Mistake?
A, B, C	0, 1, 1	not	not	–
D	2	**spam**	not	✘
E	3	**spam**	not	✘
F	2	spam	spam	–
G, H, I, J	4–8	spam	spam	–

k = 2 makes **2 mistakes** (D and E).

Step 2: repeat for every k. Doing the same check for each value gives:

k	0	1	2	3	4	5	6	7	8	9
Mistakes (out of 10)	5	4	2	2	**1**	2	3	4	4	5

Step 3: keep the best. k = 4 makes the fewest mistakes (only email F is wrong), so the learned model is:

```
IF number_of_links >= 4 THEN spam ELSE not spam
```

Nobody wrote this rule. The number 4 came from the data. If the data changed, retraining would choose a different k.

3.4 Using the model on new emails (prediction)

New email	Links	Prediction
K	7	spam
L	0	not spam
M	3	not spam

3.5 What this example shows, and what it hides

- **Shows:** the computer chose the rule, by measuring mistakes on labelled examples. This is the bottom-up idea.
- **Shows:** the model is **not perfect** even on its own training data (F is still wrong), and it can be wrong on new emails too.
- **Hides:** real models use many features (words, sender, time, …) and millions of examples, and the learned "rule" is stored as thousands or millions of numbers. That is why real models are hard to explain, even though the idea is the same.

Optional: `examples/topic03_learn_a_rule.py` runs this exact training and prints the same mistakes table and predictions.

---

4. Comparison

Question	Symbolic AI (rules)	Machine learning
Where does the knowledge come from?	Human experts write it as rules	Patterns found in labelled example data
What does it need to get started?	Experts who can state their rules clearly	Enough relevant, correctly labelled data
Can it explain an answer?	**Yes**: it lists the rules that fired (Topic 2, §4.4)	**Often difficult**: real models are many numbers. Our one-feature model is explainable only because it is tiny
What happens with an unusual case?	Fails if no rule covers it ("brittle", Topic 2 Activity D)	May handle it if similar examples were in the data; may also be **confidently wrong**
How is it updated?	People edit or add rules	Collect new data and retrain
Good fit	Clear, stable rules; answers must be justified (eligibility, regulations, safety checks)	Patterns too complex to write down (recognising images, speech, handwriting)

Key idea: neither approach is better in general. Symbolic AI is strong where rules are clear and explanations matter. Machine learning is strong where the rules are too many or too vague to write by hand **and** good data exists. Topic 5 shows systems that use **both**.

---

5. Common misunderstandings

Misunderstanding	Why it is wrong
"Machine learning means the computer understands the data."	It finds patterns that reduce mistakes. In §3 the program never knew what a "link" or "spam" is, just as the symbolic system in Topic 2 never knew what a feather is.
"A learned model is always right because it learned from real data."	Our model is wrong on email F, and a model can only be as good as its data. If the labels are wrong or the examples are unrepresentative, the model learns the wrong pattern.
"Symbolic AI is old and has been replaced by machine learning."	Rule-based systems are still widely used where exact, explainable rules are required (Topic 5). Many modern systems combine both.
"ML doesn't use rules at all."	A model *is* a kind of rule (in §3, literally a threshold rule). The difference is **who writes it**: a person, or a training process.

---

6. Learner activities

Activity A: retrain the model (trace). A new labelled email arrives: **N, 3 links, spam**. Add it to the training data (now 11 emails) and recompute the mistakes for k = 2, 3, 4 and 5. Which k is best now? What does this tell you about models and data?

Activity B: predict and explain. Using the original model (`links >= 4`), predict email P (12 links). It is actually a genuine newsletter from the user's university. Is the prediction right? Would rule R2 from §2 have handled it better? Why?

Activity C: choose an approach. For each scenario, choose **symbolic AI**, **machine learning**, or **both**, and give a reason based on the comparison table in §4.

1. Checking whether a student meets the published entry requirements for a course (minimum grades in named subjects).
2. Recognising which animal is in a photo.
3. Calculating a delivery fee from a published price list (distance bands and weight bands).
4. Recommending songs based on what millions of users have listened to.
5. A hospital system that reads X-ray images **and** must flag any result that breaks a fixed safety guideline.

Activity D: find the error. A classmate writes: "*Our spam model learned `links >= 4`. Since it made only 1 mistake out of 10, it will be about 90% correct on every future email.*" Identify the flaw in this reasoning.

---

7. Expected outcomes and answers

Activity A

With N (3 links, spam) added:

k	Mistakes	Which emails are wrong
2	**2**	D (2, not → predicted spam), E (3, not → predicted spam)
3	**2**	E (3, not → predicted spam), F (2, spam → predicted not)
4	**2**	F, N (both spam, predicted not)
5	3	F, G, N

k = 2, 3 and 4 now **tie** with 2 mistakes each, and k = 4 is no longer the clear winner. (The demo program picks the smallest k on a tie, so it would now learn `links >= 2`.) The lesson: **the learned rule depends on the data**, and one new example can change the model. Ties also show that training needs a rule for choosing between equally good options. In symbolic AI, by contrast, the rule changes only when a person edits it.

Check: add `("N", 3, "spam")` to `TRAINING` in the example program and run it.

Activity B

The model predicts **spam** (12 ≥ 4), which is **wrong**. It only looks at the number of links and has never seen a "trusted sender" feature. Rule R2 (`IF sender is in the user's contacts THEN not spam`) would handle it correctly *if* the university address is in the contacts. This shows why real systems often combine learned patterns with explicit rules (Topic 5).

Activity C

#	Best fit	Reason
1	**Symbolic**	The requirements are explicit, published rules, and the decision must be explainable to the applicant.
2	**Machine learning**	No one can write rules for every possible photo; the patterns are learned from many labelled images.
3	**Symbolic**	A published price list is exact rules; ML would add errors and remove the explanation.
4	**Machine learning**	Listening patterns across millions of users are too complex to write down, and data is plentiful.
5	**Both**	ML suits reading images; the fixed guideline is a clear rule that should be checked by rules and explained.

Accept other answers if the reason uses the §4 criteria: are the rules clear? Is an explanation required? Is there enough good data?

Activity D

The 1 mistake out of 10 was measured on the **training** emails, the same emails used to choose k. The model was *fitted* to them, so it will usually do worse on new emails, and much

worse on emails unlike the training set (e.g. Activity B's newsletter). To estimate future accuracy, you must test on **new labelled emails that were not used in training**. Ten examples is also far too few to estimate accuracy reliably.

Common mistakes

1. Counting mistakes only for spam emails and forgetting the "not spam" ones (both kinds of error count).
2. Treating the learned threshold as permanent; it changes when the data changes (Activity A).
3. Assuming ML is always the "smarter" choice, even for problems with clear, published rules (Activity C, items 1 and 3).

---

8. Summary

- **Symbolic AI:** people write the rules (top-down); explainable, but brittle and hard to maintain.
- **Machine learning:** training chooses a rule (the **model**) that makes the fewest mistakes on **labelled data** (bottom-up); flexible, but data-hungry, imperfect, and harder to explain at real-world scale.
- Accuracy measured on training data overstates how well a model will do on new data.
- The right choice depends on whether the rules are clear, whether explanations are required, and whether enough good data exists.

---

9. References

Links accessed 3 November 2026.

1. Encyclopaedia Britannica. *Machine learning.* https://www.britannica.com/technology/machine-learning
2. IBM Developer. *What is machine learning?* (contrasts programs built on previously set rules with systems that learn patterns from data). https://developer.ibm.com/articles/what-is-machine-learning/
3. Encyclopaedia Britannica. *Artificial intelligence: Expert systems.* https://www.britannica.com/technology/artificial-intelligence/Expert-systems

Note on our own material: the spam rules (§2), the ten-email dataset (§3) and all activity data are invented by the team for teaching; they are not taken from any source. The training results in §3 and Activity A were checked by running `examples/topic03_learn_a_rule.py`.

Topic 4: History: The Rise, the AI Winters and the Lessons

Level: Beginner | **Estimated time:** 35 minutes | **Status:** Drafted V2.0 (dates re-checked; awaiting peer review) **Builds on:** Topic 2 (expert systems and their limits) and Topic 3 (learning from data) | **Comes before:** Topic 5: Symbolic AI in Modern Software

Learning objectives. After this topic you will be able to:

1. Place the main events in the history of symbolic AI on a timeline.
2. Explain what an "AI winter" is, and give one *expectations* cause and one *technology* cause for each of the two main winters.
3. Use the limits you met in Topic 2 to explain why the expert-system boom ended.
4. Check a historical claim against more than one source.

---

1. Why study the history?

You now know how a rule-based system works (Topic 2) and how it differs from machine learning (Topic 3). History shows what happened when people tried to use rules as the main route to intelligent machines. It explains **why** AI today looks the way it does, and it teaches a lesson that still matters: promises about AI can run far ahead of what the technology can do.

> **AI winter:** a period in which funding for and interest in AI research fall sharply, usually after expectations were set too high and results did not match them (Wikipedia: AI winter).

> **Funding:** money for research, mostly from governments (in this period, especially the UK government, the US defence research agency DARPA, and Japan's trade ministry) and from companies.

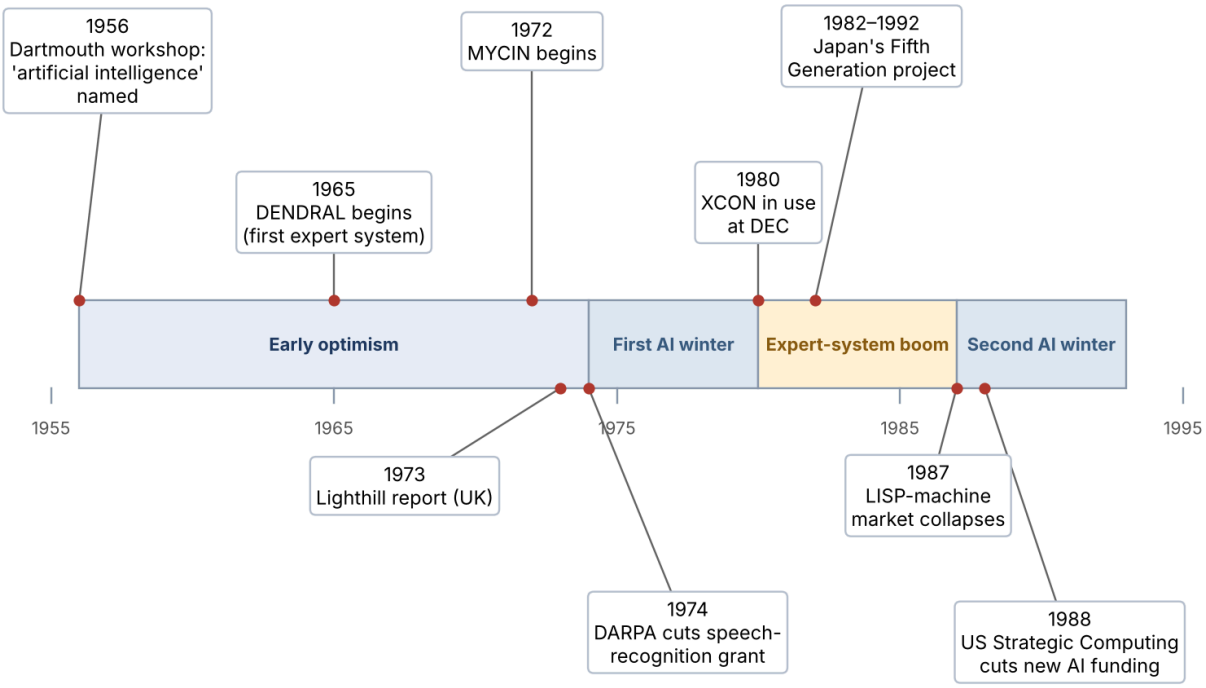
The story of symbolic AI is a cycle that happened twice:

```
big promises  →  funding  →  results fall short  →  funding is cut ("winter")
```

---

2. Timeline

Figure 4.1 Symbolic AI, 1956–1993: two cycles of promise and disappointment



Start and end years of the AI winters are approximate; historians give slightly different ranges.

Figure 4.1. The main events in this topic. (Drawn with `assets/diagram-sources/topic04_timeline.py`.)

Year(s)	Event	Phase
1956	**Dartmouth summer workshop.** John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon organise an eight-week meeting; the term "artificial intelligence" was coined for this project (Dartmouth Alumni Magazine).	Optimism
1965	Work begins on **DENDRAL** at Stanford, often called the first expert system (Britannica).	Optimism
1972	Work begins on **MYCIN** at Stanford (Topic 2, §8) (Britannica).	Optimism
1973	**The Lighthill report.** Mathematician Sir James Lighthill reviews AI for the UK's Science Research Council and concludes that "in no part of the field have the discoveries made so far produced the major impact that was then promised" (Lighthill, 1973). UK funding for AI research is sharply reduced.	First winter begins
1974	In the USA, DARPA cancels a speech-recognition research grant of about $3 million a year as it turns to research with direct military use (Wikipedia: AI winter).	First winter
≈1974–1980	**First AI winter.**	Winter
1980	**XCON** goes into everyday use at Digital Equipment Corporation (DEC) (McDermott, 1981). Companies start building expert systems.	Boom

Year(s)	Event	Phase
1982–1992	Japan's **Fifth Generation Computer Systems** project, led by the Institute for New Generation Computer Technology (ICOT) under the trade ministry (MITI), aims at computers built for knowledge-based reasoning (IPSJ Computer Museum). It builds several large parallel inference machines but falls well short of its original goals (Nature, 1992).	Boom → disappointment
1987	The market for **LISP machines** (special computers built to run AI software) collapses within about a year, as cheaper general-purpose workstations become powerful enough (Wikipedia: AI winter).	Second winter begins
1988	The US Strategic Computing Initiative stops new spending on AI (Wikipedia: AI winter).	Second winter
≈1987–1993	**Second AI winter.**	Winter

Historians give slightly different start and end years for both winters; the ranges here are the commonly cited ones.

---

3. Worked example: explaining a historical event step by step

History questions often ask **"why did X happen?"**. A useful method is to separate the causes into two kinds, then link them to the effect.

Question: Why did the first AI winter happen?

Step 1: What was promised? Early researchers expected that machines with general, human-like intelligence would arrive within a generation. The Dartmouth proposal itself hoped for major progress in a single summer.

Step 2: What was delivered? Programs that worked well on small, carefully chosen problems ("toy problems") but failed on real-world ones such as understanding ordinary language.

Step 3: Why the gap? (technology cause) Lighthill identified the **combinatorial explosion**: as a problem grows, the number of possibilities a program must consider grows so fast that the computer cannot check them all. Computers of the time were also slow, with very little memory.

> **Combinatorial explosion:** the very rapid growth in the number of possible combinations as a problem gets larger. *Example:* a password of 2 digits has 100 possibilities; 4 digits, 10,000; 8 digits, 100,000,000. Each extra digit multiplies the work by 10.

Step 4: Who reacted, and how? (expectations cause → effect) Funders compared the promises with the results. The UK government (after the Lighthill report) and DARPA in the

USA cut funding.

Step 5: Write the explanation.

> The first AI winter happened because early promises of human-like intelligence (**expectations**) could not be met by the computers and methods of the time, which ran into the combinatorial explosion on real problems (**technology**). When reviews such as the Lighthill report (1973) made the gap clear, governments in the UK and USA cut funding (**effect**).

You will use the same method in Activity B.

---

4. The second winter: the limits of rules

This part connects directly to Topic 2. The expert-system boom of the 1980s ran into exactly the limits you met in the animal identifier:

Limit you saw in Topic 2	What happened in industry
The system knows only what its rules cover (Activity Part D)	Systems failed on unusual cases; they were described as **brittle**
Adding a rule can create a conflict (Activity Part F)	Large rule bases became very costly to update and maintain. XCON, for example, needed expensive rewriting (Middlesex University notes)
No learning (Topic 3)	Every change needed human experts and programmers

At the same time, the special hardware that AI companies sold (LISP machines) was overtaken by cheaper ordinary computers, and that whole industry disappeared within about a year (Wikipedia: AI winter).

What came next

Rule-based AI did **not** disappear. It stopped being sold as a path to human-like intelligence and became an ordinary software technique (Topic 5). Meanwhile, research moved towards approaches that **learn from data** (Topic 3), which later benefited from much faster computers and much more data.

---

5. Common misunderstandings

Misunderstanding	What is actually true
"During an AI winter, all AI research stopped."	Funding and public interest fell sharply, but research continued at a smaller scale. Some expert systems, such as XCON, stayed in use.

Misunderstanding	What is actually true
"Symbolic AI failed, so it was abandoned."	It failed to deliver **general** intelligence, but rule-based methods are still used widely (Topic 5).
"The winters were caused only by slow computers."	Over-promising mattered just as much. Both winters followed periods when expectations ran far ahead of results.
"History dates are fixed facts."	Exact dates and amounts can differ between sources. In V2.0 we found that one source dated the Fifth Generation project to 1981–1991, while the project's own museum record and a 1992 *Nature* report give 1982–1992 (see Activity D).

---

6. Learner activities

Activity A: put events in order. Arrange these five events in time order: Fifth Generation project starts · Dartmouth workshop · LISP-machine market collapses · Lighthill report · XCON enters use.

Activity B: explain a decision (use the method in §3). Explain why the second AI winter happened. Your answer must include at least one **expectations** cause, one **technology** cause, and a link to a limit you saw in Topic 2.

Activity C: predict. It is 1985. A company asks you to build an expert system that answers *any* customer question about *any* of its 5,000 products. Using what you learned in Topic 2 and this topic, predict two problems the project would face.

Activity D: check a claim (find the error). A website says: "*Japan's Fifth Generation project ran from 1981 to 1991 and cost $850 million.*" Using the sources in §2 and the references below, which parts can you confirm, and which parts disagree with other sources? What should a careful writer do?

---

7. Expected outcomes and answers

Activity A

Dartmouth workshop (1956) → Lighthill report (1973) → XCON enters use (1980) → Fifth Generation project starts (1982) → LISP-machine market collapses (1987).

Activity B (model answer)

> In the 1980s, companies and governments expected expert systems, and projects such as Japan's Fifth Generation programme, to deliver far more than they did (**expectations**). In practice, rule-based systems were brittle: like the animal identifier in Topic 2, they failed on cases their rules did not cover, and every new rule risked conflicts, so large systems became very expensive to maintain (**technology / link to Topic 2**). At the same time, cheaper general-purpose computers replaced special LISP

> machines, and that market collapsed in 1987 (**technology**). As funding was cut, interest fell, which is the second AI winter.

A full-credit answer names one expectations cause, one technology cause, and explicitly mentions a Topic 2 limit (gap in the knowledge base, rule conflict, or no learning).

Activity C

Any two of the following, with a reason:

- **Gaps (brittleness):** customers will ask questions no rule covers, and the system will fail or give a wrong answer (Topic 2, Part D).
- **Combinatorial growth and maintenance:** 5,000 products need thousands of rules; each new or changed product needs new rules, and new rules can conflict with old ones (Topic 2, Part F).
- **Expert time:** experts must state every rule, and they may disagree with each other. (XCON's developers met disagreements between DEC's own experts (Wikipedia: XCON).)
- **Language:** understanding free-form questions was one of the problems that defeated early AI (§3).

Activity D

- **Confirmed:** it was a Japanese government project aimed at knowledge-based computing, and it fell short of its goals (IPSJ; *Nature*, 1992).
- **Disagree:** the **dates**: the IPSJ Computer Museum gives 1982–1992, and *Nature* (March 1992) says the ten-year project launched in 1982 and was extended by a year. The **cost**: IPSJ gives about ¥54 billion; *Nature* reports "over $400 million" after ten years. The $850 million figure appears in a secondary source but is not supported by these two.
- **What to do:** prefer the project's own records and reports from the time; when sources disagree, give the range or say "about", and cite the source used. (This is what we changed in V2.0 of this topic.)

Common mistakes

1. Listing only "computers were slow" for both winters; the expectations cause is required.
2. Saying symbolic AI "ended" in the 1990s (see §5).
3. Treating any single website as enough to confirm a date or amount.

---

8. Summary

- AI's early history went through **two cycles** of big promises, funding, disappointment and funding cuts (**AI winters**, ≈1974–1980 and ≈1987–1993).

- The first winter came from over-promising and the **combinatorial explosion** on slow computers; the Lighthill report (1973) was a turning point in the UK.
- The second winter came from the limits of expert systems (**brittle**, costly to maintain, unable to learn) and the collapse of the LISP-machine market.
- Rules did not disappear; they became ordinary software tools, while research moved towards learning from data.

---

9. References

Links accessed 31 October 2026; Lighthill, IPSJ, *Nature* and McDermott sources added 3 November 2026.

1. Hertog, J. *The Birth of Artificial Intelligence.* Dartmouth Alumni Magazine. https://dartmouthalumnimagazine.com/articles/birth-artificial-intelligence
2. Encyclopaedia Britannica. *Artificial intelligence: Expert systems.* https://www.britannica.com/technology/artificial-intelligence/Expert-systems
3. Lighthill, J. (1973). *Artificial Intelligence: A General Survey.* Science Research Council (UK). Copy hosted by the University of Edinburgh AIAI. https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf
4. McDermott, J. (1981). R1: The formative years. *AI Magazine*, 2(2). https://aitopics.org/doc/journals:E2B8C719
5. Information Processing Society of Japan, Computer Museum. *Fifth Generation Computer Systems (FGCS) project.* https://museum.ipsj.or.jp/en/computer/other/0002.html
6. *Nature* (26 March 1992). Japan stubs its toes on fifth-generation computer. *Nature* 356, 273. https://www.nature.com/articles/356273b0.pdf
7. Huyck, C. *Configuration with R1/XCon.* BIS2040 course notes, Middlesex University. https://cwa.mdx.ac.uk/bis2040/lect6ES2/xCon.html
8. Wikipedia. *AI winter* (DARPA 1974, LISP-machine collapse 1987, Strategic Computing 1988). https://en.wikipedia.org/wiki/AI_winter
9. Wikipedia. *Xcon.* https://en.wikipedia.org/wiki/Xcon

Still to confirm (see `docs/review-log.md`): the DARPA grant (1974), LISP-machine collapse (1987) and Strategic Computing cut (1988) are supported only by Wikipedia. The combinatorial-explosion password example is the team's own illustration.

Topic 5: Symbolic AI in Modern Software

Level: Beginner | **Estimated time:** 40 minutes | **Status:** Drafted V2.0 (awaiting peer review) **Builds on:** Topics 1–4 | **Followed by:** Module test bank

Learning objectives. After this topic you will be able to:

1. Identify rule-based reasoning inside everyday software, and write it as rules or as a decision table.
2. Trace a rule-based game character through a sequence of events.
3. Explain what a knowledge graph is and follow links in it to answer a question.
4. Describe how a neuro-symbolic system combines learning with rules, using one real example.

---

1. Rules did not disappear

Topic 4 ended with the AI winters. But the idea of **IF–THEN rules plus a program that applies them** never went away. It became such an ordinary part of software that it is rarely called "AI" any more. This topic shows four places you meet it.

1.1 Online forms

When you leave a required box empty on a website form and press *Submit*, the browser blocks the form and shows a message such as "Please fill out this field". This is a rule built into web browsers: a field marked `required` must not be empty, and the browser checks it before the form is sent (MDN Web Docs: Client-side form validation).

```
IF field is required AND field is empty THEN block submission AND show message
```

This is a Boolean condition from Topic 1, applied automatically by software.

1.2 Business rules and decision tables

> **Business rules engine:** software that stores an organisation's decision rules separately from the rest of the program, so that staff can change the rules without rewriting the program.

Salesforce, for example, sells a Business Rules Engine. Its training material describes it as automating complex decisions, organising rules in tools such as **decision tables**, and lists uses such as checking **eligibility** for discounts and public benefits and calculating insurance quotes (Salesforce Trailhead: Get to Know Business Rules Engine).

> **Decision table:** a table in which each row is one rule. The columns on the left are conditions, and the column on the right is the result. It is a compact way to write many

> IF–THEN rules, and it makes it easy to check that **every case is covered exactly once**.

Example: a bus-pass discount *(our own illustration)*

Rule	Is a student?	Age ≥ 65?	Result
D1	Yes	(any)	50% discount
D2	No	Yes	50% discount
D3	No	No	Full price

Each row is an IF–THEN rule (Topic 1). The table as a whole is the condition `student OR age ≥ 65`. In Topic 2's terms, the table is a small **knowledge base** and the business rules engine plays the role of the **inference engine**.

1.3 Characters in video games

Many computer-controlled characters follow a **finite state machine**: the character is always in one **state** (such as *Wander*, *Attack* or *Flee*) and changes state when a rule's condition becomes true. Game-design teaching material describes this as the most common pattern for game AI, because it is easy to draw, program and debug (*Principles of Computer Game Design and Implementation*, Lecture 23).

> **State:** what the character is currently doing. **Transition:** a rule that moves the character from one state to another: `IF (current state) AND (condition) THEN (new state)`.

The lecture's example guard uses these transition rules:

Rule	IF current state is…	AND…	THEN new state
G1	Wander	player is seen	Attack
G2	Attack	health is low	Flee
G3	Attack	no player is present	Wander
G4	Flee	no player is present	Wander

If no rule matches, the character **stays in its current state**.

---

2. Worked example: tracing a game guard

The guard starts in **Wander** with full health. Here is what happens over five moments of play:

Moment	What is happening	Current state	Rule that matches	New state
1	Player not visible	Wander	none (G1 needs "player is seen")	**Wander**
2	Player comes into view	Wander	**G1**	**Attack**
3	Player still visible; guard's health falls to low	Attack	**G2**	**Flee**
4	Player still visible	Flee	none (G4 needs "no player present")	**Flee**
5	Player leaves	Flee	**G4**	**Wander**

What to notice:

- In each moment the guard checks only the rules for its **current** state, just as an inference engine checks which rules have a true IF part (Topic 2).
- At moment 4 the player is visible, but the guard does *not* attack again. No rule says "Flee + player seen → Attack". The behaviour is entirely decided by the rules people wrote, including what they left out.
- Because each state change is caused by a named rule, a game designer can explain exactly *why* the guard ran away (rule G2), which is the explainability strength from Topic 2.

---

3. Knowledge graphs: facts as a network

> **Knowledge graph:** a large collection of facts stored as **things** (people, places, objects) connected by named **relationships**. Each fact has the form *thing* → *relationship* → *thing*.

In 2012 Google introduced its Knowledge Graph to help its search engine understand "things, not strings". At launch it held over 500 million objects and more than 3.5 billion facts about them and their relationships, which lets a search tell, for example, the Taj Mahal monument apart from a musician with the same name (Google: Introducing the Knowledge Graph).

Worked example: following links with a rule

A tiny knowledge graph *(our own example)*:

```
Registan   --located in-->  Samarkand
Samarkand  --located in-->  Uzbekistan
Uzbekistan --located in-->  Central Asia
Registan   --is a-->        public square
```

The graph does **not** store "Registan is located in Central Asia". But one rule lets the system work it out:

```
L1: IF X is located in Y AND Y is located in Z THEN X is located in Z
```

Question: Is the Registan located in Central Asia?

Step	Facts used	Rule	New fact
1	Registan → Samarkand; Samarkand → Uzbekistan	L1	Registan is located in Uzbekistan
2	Registan → Uzbekistan (new); Uzbekistan → Central Asia	L1	**Registan is located in Central Asia**

This is **forward chaining** (Topic 2) on a network of facts. Large knowledge graphs do the same thing with billions of facts.

---

4. Neuro-symbolic AI: learning and rules together

> **Neuro-symbolic AI:** a system that combines a **neural network** (a type of machine-learning model, Topic 3) with **symbolic reasoning** (rules and logic, Topic 2), so that each covers the other's weaknesses.

Case study: AlphaGeometry (Google DeepMind, January 2024) (DeepMind blog)

Part	What it does	Which topic it comes from
Symbolic deduction engine	Applies geometry rules step by step to derive new facts about the diagram, like forward chaining	Topic 2
Language model (learned from data)	When the engine gets stuck, suggests one useful new construction (a point, line or circle) to add; then the engine continues	Topic 3

- **Result:** on 30 olympiad geometry problems, AlphaGeometry solved 25 within the time limit. The previous best automatic method solved 10, and the average human gold medallist solves 25.9.
- **Why combine them:** DeepMind describes the language model as providing fast "intuition" and the symbolic engine as providing rigorous reasoning that can be checked.

This sums up the whole module: rules give reasoning that can be **explained and checked** (Topic 2), and learning gives **flexibility** that hand-written rules lack (Topic 3).

---

5. Common misunderstandings

Misunderstanding	What is actually true
"If it isn't machine learning, it isn't AI."	Rule-based reasoning is the original form of AI (Topics 2 and 4) and is still used. It is simply less often *called* AI now.
"A game character that seems clever must be learning."	Most follow hand-written state rules (§1.3). Their "intelligence" is the designer's rules, as the moment-4 example in §2 shows.
"A knowledge graph knows only the facts typed into it."	It can also derive new facts by applying rules to the links (§3), just like an inference engine.
"Neuro-symbolic systems replaced rules with learning."	They keep both: in AlphaGeometry the symbolic engine does the actual proof.

---

6. Learner activities

Activity A: trace the guard (predict). Using rules G1–G4, the guard starts in **Wander**. Predict its state after each moment:

1. Player seen, health full
2. Player not present, health full
3. Player seen, health low
4. Player seen, health low
5. Player not present, health low

Activity B: find the error in a decision table. A delivery app uses this table:

Rule	Distance (km)	Fee
F1	less than 5	€2
F2	more than 5 and at most 10	€4
F3	more than 10	€6

A customer exactly 5 km away gets an error. Find the problem and correct the table. (Hint: Topic 1, Task B.)

Activity C: follow the graph. Add these facts to the graph in §3: `Bukhara --located in--> Uzbekistan` and `Ark of Bukhara --located in--> Bukhara`. Show, step by step with rule L1, how the system concludes that the Ark of Bukhara is in Central Asia.

Activity D: spot the rules in an app you use. Choose an app or website you use often.

1. Find **two** decisions it probably makes using rules. Write each as an IF–THEN rule.
2. Find **one** feature that probably uses machine learning, and explain why it would be hard to write as rules.
3. Is there a feature that seems to use **both**? Describe it in one sentence.

---

7. Expected outcomes and answers

Activity A

Moment	Rule	State after
1	G1	Attack
2	G3	Wander
3	G1 (in Wander, only G1 can apply; health does not matter in Wander)	Attack
4	G2	Flee
5	G4	Wander

The trap is moment 3: the guard is in **Wander**, so G2 (which needs state *Attack*) cannot fire yet. It attacks first, then flees at moment 4. Checking the **current state** first is the key step.

Activity B

The table has a **gap**: 5 km is not "less than 5" (F1) and not "more than 5" (F2), so no rule matches. This is the same strict-boundary issue as 18 °C in Topic 1. One correct table:

Rule	Distance (km)	Fee
F1	less than 5	€2
F2	**at least 5** and at most 10	€4
F3	more than 10	€6

Check: every distance now falls into **exactly one** row (no gaps, no overlaps). Making F1 "at most 5" is also correct, as long as F2 then starts at "more than 5".

Activity C

Step	Facts used	Rule	New fact
1	Ark of Bukhara → Bukhara; Bukhara → Uzbekistan	L1	Ark of Bukhara is located in Uzbekistan
2	Ark of Bukhara → Uzbekistan; Uzbekistan → Central Asia	L1	**Ark of Bukhara is located in Central Asia**

Activity D

There is no single correct answer. Check each response against these criteria:

Part	A good answer…	Example (food-delivery app) ***(our illustration)***
1	States clear true/false conditions and a result	`IF order total < minimum THEN disable "Checkout"`; `IF restaurant is closed THEN hide "Order now"`

Part	A good answer…	Example (food-delivery app) *(our illustration)*
2	Names a feature that depends on patterns in lots of data	"Recommended for you": depends on millions of past orders; too many patterns to write as rules
3	Shows a learned part plus a rule part	Delivery-time estimate learned from past trips, **plus** a rule that adds 10 minutes IF it is raining

Common mistakes

1. Activity A: applying a rule for the wrong state (e.g. G2 while still in Wander).
2. Activity B: fixing the gap but creating an overlap (e.g. F1 "at most 5" *and* F2 "at least 5" both match 5 km).
3. Activity D: calling any automatic feature "machine learning". A fixed IF–THEN check is a rule.

---

8. Module summary

Topic	Big idea
1	Computers decide by testing true/false conditions with IF–THEN rules
2	Symbolic AI = a knowledge base of rules + an inference engine that chains them, and it can explain itself
3	Machine learning chooses its rule from labelled data instead; each approach suits different problems
4	Over-promising and the limits of rules caused two AI winters
5	Rules live on in forms, business rules, games, knowledge graphs and neuro-symbolic systems

Check yourself: the module test bank has questions on every topic, with answers.

---

9. References

Links accessed 31 October and 3 November 2026.

1. MDN Web Docs (Mozilla). *Client-side form validation.* https://developer.mozilla.org/en-US/docs/Learn/HTML/Forms/Form_validation
2. Salesforce Trailhead. *Get to Know Business Rules Engine.* https://trailhead.salesforce.com/content/learn/modules/business-rules-engine/get-to-know-business-rules-engine
3. *Principles of Computer Game Design and Implementation*, Lecture 23 (finite state machines; Wander / Attack / Flee example). University of Liverpool course material. https://intranet.csc.liv.ac.uk/~xiaowei/game_materials/lecture23.pdf
4. Singhal, A. (16 May 2012). *Introducing the Knowledge Graph: things, not strings.* Google. https://blog.google/products/search/introducing-knowledge-graph-things-not/

5. Google DeepMind (17 January 2024). *AlphaGeometry: An Olympiad-level AI system for geometry.* https://deepmind.google/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/

Our own material: the bus-pass table, the delivery-fee table, the five-moment guard trace, the Registan/Bukhara graph, rule L1 and the food-delivery examples are written by the team for teaching. The guard's four transition rules follow the lecture's example; the trace and activity are ours.

Module Test Bank

Status: Drafted V2.0 (answers checked against the topic files; awaiting peer review) | **Used by:** Study Guide end-of-chapter questions, slide quick-checks, video "pause and try" moments

Each question is tagged with the **learning outcome** (LO, see module outline), the **topic and section** it tests, and its **type**:

- **Recall:** remember a definition or fact
- **Apply:** use an idea on a new case (trace, calculate, write a rule)
- **Analyse:** find an error, predict an outcome, compare or justify

Answers are in the second half of this file so learners can try first. Every answer refers back to the section that teaches it. **If an answer changes, update the topic file too** (and vice versa).

Part 1: Questions

Topic 1: Software and basic logic

Q1 *(LO1 · Recall · Topic 1 §2.1)* Which of these is **software**? a) a keyboard b) a temperature sensor c) an operating system d) a memory chip

Q2 *(LO1 · Recall · Topic 1 §2.2)* Which property must every algorithm have? a) It must be written in Python b) It must finish after a finite number of clear steps c) It must use AND d) It must run on a phone

Q3 *(LO2 · Apply · Topic 1 §2.3)* `raining = False`, `snowing = True`. What is `raining OR snowing`? What is `NOT snowing`?

Q4 *(LO2 · Apply · Topic 1 §5 Task B)* A fridge alarm rule is `IF T > 5 THEN alarm ON ELSE alarm OFF`. Complete the result for T = 4, 5 and 6.

Q5 *(LO2 · Analyse · Topic 1 §5 Task D)* A library app uses `can_borrow = (has_card) OR (no_overdue_books)`. Give one situation where this rule gives the wrong result, and correct the rule.

Topic 2: Rules and inference

Questions Q6–Q11 use the knowledge base from Topic 2, §4.1:

Rule	IF …	THEN …
R1	has feathers	is a bird
R2	gives milk	is a mammal
R3	is a bird AND cannot fly AND swims	is a penguin

Rule	IF …	THEN …
R4	is a bird AND can fly AND hunts at night	is an owl
R5	is a mammal AND eats meat AND has black stripes	is a tiger
R6	is a mammal AND has hooves AND has black stripes	is a zebra

Q6 *(LO3 · Recall · Topic 2 §3)* Which part of a rule-based system changes for every new case? a) the knowledge base b) working memory c) the inference engine d) the rules

Q7 *(LO4 · Apply · Topic 2 §4.3)* Trace forward chaining for the starting facts `has feathers`, `can fly`, `hunts at night`. Give the rule fired and new fact in each cycle, and the cycle in which the engine stops.

Q8 *(LO4 · Analyse · Topic 2 §6 Part E)* A student's trace for `gives milk`, `has hooves`, `has black stripes` says: "Cycle 1: R6 fires → is a zebra." What is wrong?

Q9 *(LO4 · Apply · Topic 2 §4.5)* Using backward chaining with the goal *is an owl?*, the system knows only `has feathers`. List the questions it must ask the user.

Q10 *(LO3 · Analyse · Topic 2 §6 Part D)* The starting facts are `gives milk` and `swims`. What does the system conclude? Is this a fault in the inference engine? Explain.

Q11 *(LO3 · Recall · Topic 2 §4.4)* Why is the ability to answer "why?" useful in an expert system such as MYCIN?

Topic 3: Symbolic AI vs machine learning

Questions Q13–Q14 use the training data from Topic 3, §3.1:

Email	A	B	C	D	E	F	G	H	I	J
Links	0	1	1	2	3	2	4	5	6	8
Label	not	not	not	not	not	spam	spam	spam	spam	spam

Q12 *(LO5 · Recall · Topic 3 §1)* In machine learning, what is **labelled data**?

Q13 *(LO5 · Apply · Topic 3 §3.3)* How many training mistakes does the rule `IF links >= 5 THEN spam` make? Which emails are wrong?

Q14 *(LO5 · Analyse · Topic 3 §6 Activity D)* A team reports: "Our model is 90% accurate" after measuring it on the same 10 emails used to choose k. Why is this claim weak, and what should they do instead?

Q15 *(LO5 · Analyse · Topic 3 §4)* A bank must explain to every customer exactly why a loan application was refused, using its published criteria. Symbolic AI, machine learning, or both? Justify.

Q16 *(LO5 · Analyse · Topic 3 §4)* Give **one** advantage and **one** disadvantage of machine learning compared with symbolic AI.

Topic 4: History and AI winters

Q17 *(LO6 · Recall · Topic 4 §2)* Put in order: XCON enters use · Lighthill report · Dartmouth workshop · LISP-machine market collapses.

Q18 *(LO6 · Apply · Topic 4 §3)* Using a 3-digit lock (digits 0–9) and then a 6-digit lock, explain the **combinatorial explosion** in one or two sentences with numbers.

Q19 *(LO6 · Analyse · Topic 4 §4)* Explain how the "rule conflict" problem you saw in Topic 2 (Part F) helps explain the end of the expert-system boom.

Q20 *(LO6 · Analyse · Topic 4 §5)* True or false, with a reason: "During the AI winters, all AI research and all expert systems stopped."

Topic 5: Symbolic AI in modern software

Questions Q21–Q22 use the guard rules from Topic 5, §1.3:

Rule	IF current state is…	AND…	THEN new state
G1	Wander	player is seen	Attack
G2	Attack	health is low	Flee
G3	Attack	no player is present	Wander
G4	Flee	no player is present	Wander

Q21 *(LO7 · Apply · Topic 5 §2)* The guard is in **Attack**. The player stays visible and the guard's health stays full for three moments. What state is the guard in after each moment? Why?

Q22 *(LO7 · Analyse · Topic 5 §2)* A designer wants the guard to attack again if it is fleeing and its health has recovered. Write the missing transition rule.

Q23 *(LO7 · Analyse · Topic 5 §6 Activity B)* A cinema uses this table: *Child: age under 12 → €5; Adult: age over 12 and under 65 → €10; Senior: age 65 or over → €7.* Which age is not covered? Correct the table.

Q24 *(LO7 · Apply · Topic 5 §3)* Facts: `Bishkek --located in--> Kyrgyzstan`, `Kyrgyzstan --located in--> Central Asia`. Using rule L1, what new fact can be derived? Show the step.

Q25 *(LO7 · Analyse · Topic 5 §4)* In AlphaGeometry, which part does the step-by-step proving, and which part suggests new constructions? Which of these parts comes from symbolic AI?

Part 2: Answers and explanations

Q	Answer	Explanation / where taught
1	**c**	An operating system is instructions; the others are physical hardware (Topic 1 §2.1).
2	**b**	Algorithms are finite, clear, step-by-step procedures; the language and device do not matter (Topic 1 §2.2).
3	`raining OR snowing` = **True**; `NOT snowing` = **False**	OR needs at least one true part; NOT flips the value (Topic 1 §2.3).
4	T = 4 → OFF; T = 5 → **OFF**; T = 6 → ON	`5 > 5` is false: "greater than" is strict, the same trap as 18 °C (Topic 1 §6 Task B).
5	A person with **no card** but no overdue books can borrow (OR is true). Correct rule: `can_borrow = (has_card) AND (no_overdue_books)`	Same error as the OR login rule (Topic 1 §6 Task D). Any row where exactly one part is true shows the fault.
6	**b**	Working memory holds this case's facts; the knowledge base (rules) stays the same (Topic 2 §3).
7	Cycle 1: **R1** → is a bird. Cycle 2: **R4** (is a bird AND can fly AND hunts at night) → is an owl. Cycle 3: no new fact → **stop**	Checked with `examples/topic02_mini_rule_engine.py`.
8	R6 cannot fire in cycle 1 because `is a mammal` is not yet known. Correct: cycle 1 **R2** → is a mammal; cycle 2 **R6** → is a zebra; cycle 3 stop.	New facts become available only in the next cycle (Topic 2 §6 Part E).
9	Only R4 concludes *owl*, so it needs `is a bird` (proved by R1 from `has feathers` ✔), `can fly`, `hunts at night`. It asks: **"Can it fly?"** and **"Does it hunt at night?"**	Backward chaining asks only about facts needed for the goal (Topic 2 §4.5).
10	Only **"is a mammal"** (R2). No rule uses `swims` with `is a mammal`. Not an engine fault: a **gap in the knowledge base**.	The system knows only what its rules cover (brittleness, Topic 2 §6 Part D).
11	It shows the rules that led to the conclusion, so a doctor can **check each step** and decide whether to trust it.	Topic 2 §4.4 and §8.
12	Examples that each come with their **correct answer** (label), e.g. emails marked spam / not spam.	Topic 3 §1.
13	**2 mistakes**: F (2 links, spam → predicted not) and G (4 links, spam → predicted not).	All others match. Checked with `examples/topic03_learn_a_rule.py` (k = 5 row).

Q	Answer	Explanation / where taught
14	The model was fitted to those same emails, so accuracy on them **overstates** future accuracy; 10 examples are also too few. Test on **new labelled emails not used in training**.	Topic 3 §7 Activity D.
15	**Symbolic AI** (possibly with ML used only for other tasks). The criteria are published rules and every refusal must be explained; rules can list exactly which criterion failed.	Topic 3 §4: "answers must be justified".
16	*Advantage (any one):* learns patterns too complex to write by hand; can be retrained when data changes. *Disadvantage (any one):* needs lots of good labelled data; hard to explain at real-world scale; can be confidently wrong.	Topic 3 §4.
17	Dartmouth (1956) → Lighthill (1973) → XCON (1980) → LISP-machine collapse (1987)	Topic 4 §2, Figure 4.1.
18	3 digits: 10 × 10 × 10 = **1,000** combinations; 6 digits: **1,000,000**. Adding 3 digits multiplied the work by 1,000. This rapid growth is the combinatorial explosion.	Topic 4 §3.
19	As rule bases grew to thousands of rules, every new rule could clash with old ones (like R5 and R6 both firing). Finding and fixing conflicts made large systems expensive to maintain, one reason companies lost faith in expert systems in the late 1980s.	Topic 4 §4 links Topic 2 Part F to the second winter.
20	**False.** Funding and interest fell sharply, but research continued at a smaller scale and some systems (e.g. XCON) stayed in use.	Topic 4 §5.
21	**Attack, Attack, Attack.** G2 needs low health and G3 needs no player present; neither is true, so no rule fires and the guard stays in its current state.	Topic 5 §1.3: "If no rule matches, the character stays in its current state."
22	`G5: IF current state is Flee AND health is not low AND player is seen THEN Attack` (accept equivalent wording that names the current state, the health condition and the new state).	Topic 5 §2, moment 4 shows the missing behaviour.

Q	Answer	Explanation / where taught
23	Age **12** is not covered (not "under 12", not "over 12"). Fix: Adult = **12 or over** and under 65 (or Child = 12 or under).	Gap at a boundary, as in Topic 5 Activity B and Topic 1 Task B.
24	Bishkek → Kyrgyzstan and Kyrgyzstan → Central Asia, so by L1: **Bishkek is located in Central Asia**.	Topic 5 §3 worked example.
25	The **symbolic deduction engine** does the proving (this part is symbolic AI); the **language model** suggests constructions (this part is learned from data).	Topic 5 §4.

Coverage check

Learning outcome	Questions	Types included
LO1 Software, algorithms, conditions	Q1, Q2	Recall
LO2 IF–THEN rules and truth tables	Q3, Q4, Q5	Apply, Analyse
LO3 Parts of a rule-based system	Q6, Q10, Q11	Recall, Analyse
LO4 Tracing forward/backward chaining	Q7, Q8, Q9	Apply, Analyse
LO5 Symbolic AI vs ML	Q12–Q16	Recall, Apply, Analyse
LO6 History and AI winters	Q17–Q20	Recall, Apply, Analyse
LO7 Modern uses	Q21–Q25	Apply, Analyse

Totals: 25 questions: 6 recall, 8 apply, 11 analyse.

Glossary

Every technical term in the module, with the **one definition we use everywhere** (Markdown, Study Guide, slides and videos). If a definition changes, change it here first, then in the topic file and every resource that uses it.

The **Topic** column shows where the term is first introduced.

Term	Definition used in this module	Topic
AI winter	A period in which funding for and interest in AI research fall sharply, usually after expectations were set too high.	4
Algorithm	A step-by-step procedure that solves a problem in a finite number of clear steps.	1
AND	Boolean operator; true only when **both** parts are true.	1
Backward chaining	Inference that starts from a **goal** and works back to the facts needed to prove it (goal-driven).	2
Boolean condition (or **condition**)	A question or statement that can only be true or false, e.g. `T < 18`.	1
Boolean logic	Logic in which every statement is true or false, combined with AND, OR and NOT.	1
Brittle	Describes a system that fails badly on cases its rules do not cover.	2
Business rules engine	Software that stores an organisation's decision rules separately from the rest of the program, so they can be changed without rewriting the program.	5
Combinatorial explosion	The very rapid growth in the number of possible combinations as a problem gets larger.	4
Decision table	A table in which each row is one rule: conditions on the left, the result on the right.	5
Expert system	A program that uses AI methods, usually a knowledge base of rules and an inference engine, to solve problems in one specialised area.	2
Fact	A statement taken as true for the current case, e.g. `has feathers`.	2
Feature	A measurable property of an example that an ML program may use, e.g. number of links.	3
Fire (a rule)	What happens when the inference engine finds a rule's IF part true and adds its THEN part to working memory.	2
Forward chaining	Inference that starts from known facts and keeps firing rules until nothing new can be added (data-driven).	2

Term	Definition used in this module	Topic
Hardware	The physical parts of a computer.	1
Inference	Working out a new fact from facts already known.	2
Inference engine	The program that repeatedly finds rules whose IF part is true and fires them.	2
Knowledge base	The stored collection of rules for one subject area; the same for every case.	2
Knowledge graph	A large collection of facts stored as things connected by named relationships.	5
Label / **labelled data**	The correct answer attached to an example / examples that each have their correct answer.	3
Machine learning (ML)	AI in which software learns patterns from data instead of following rules written in advance.	3
Model	The result of training: the learned "rule" used to answer new cases.	3
Neuro-symbolic AI	A system combining a neural network (learning) with symbolic reasoning (rules).	5
NOT	Boolean operator; flips true to false and false to true.	1
OR	Boolean operator; true when **at least one** part is true (including both).	1
Prediction	A model's answer for a new, unlabelled example.	3
Pseudocode	Program-like steps in plain language; used to plan an algorithm, cannot be run.	1
Rule	An IF–THEN statement: if all conditions (the IF part) are true, the conclusion (the THEN part) is added or carried out.	1, 2
Rule conflict	When two rules can both fire and give contradictory conclusions.	2
Software	The instructions (programs) that tell a computer what to do.	1
State / **transition**	What a rule-based character is currently doing / a rule that moves it to another state.	5
Symbol	A name or label that stands for something, e.g. `is a bird`. The computer does not know its meaning.	2
Symbolic AI	AI that represents knowledge as human-readable symbols and reasons by applying rules to them.	1, 2
Threshold rule	A rule that classifies by comparing one value with a cut-off, e.g. `links >= 4`.	3
Training	The process in which an ML program searches for the pattern that best fits labelled data.	3

Term	Definition used in this module	Topic
Truth table	A table listing every combination of true/false inputs with the result for each.	1
Working memory	The facts about the current case: starting facts plus facts the engine adds.	2

Consistency decisions made in V2.0:

- We say **"inference engine"**, never "rules engine", when talking about expert systems; "business rules engine" is used only for the commercial software in Topic 5.
- We say **"working memory"**, not "fact base" or "database", for the facts about the current case.
- Each topic writes **"IF part"** and **"THEN part"**; *condition*/*antecedent* and *conclusion*/*consequent* appear once in Topic 2 as alternative names only.
- **symbolic AI** is lower case in running text and capitalised only at the start of a sentence, in headings and in titles (the same as "machine learning").